

Nonlinear and Adaptive Flight Control

Eltjo Rietman and Tycho Rietman

Based on lecture slides of AE4311 Nonlinear and Adaptive Flight Control, TU Delft Faculty of Aerospace Engineering

Student-written notes covering nonlinear dynamic inversion, aircraft attitude and rate control, time-scale separation, adaptive and incremental NDI, and backstepping control.

Based primarily on the lecture material of TU Delft AE4311. These notes are unofficial and may contain errors.

Covered topics:

1. *Nonlinear Dynamic Inversion*
2. *Aircraft Flight Dynamics for NDI*
3. *Aircraft Angular-Rate control with NDI*
4. *Time-Scale-Separated NDI and Sideslip Control*
5. *Adaptive Nonlinear Dynamic Inversion (ANDI)*
6. *Incremental Nonlinear Dynamic Inversion (INDI)*
7. *Adaptive INDI*
8. *Backstepping Control Design*

Use of generative AI: Generative AI tools were used for language correction and restructuring of author-written material. The authors remain responsible for the technical content, derivations, interpretation, and final verification.

1. Nonlinear Dynamic Inversion

1.1 Motivation for NDI

Nonlinear Dynamic Inversion (NDI) is a model-based control method for nonlinear dynamical systems. Rather than mapping tracking error directly to actuator commands, NDI uses the nonlinear plant model to determine the input required to impose a desired input-output response.

1.2 NDI in Controller-Companion Form

The simplest setting is the controller-companion form, in which the control input appears only in the final state equation.

$$\frac{d}{dt} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} x_2 \\ x_3 \\ \dots \\ x_n \\ b(x) + a(x)u \end{bmatrix}$$

Because the input appears explicitly in the final equation, the dynamics can be inverted algebraically.

$$u = a^{-1}(x)[v - b(x)]$$

which gives $\dot{x}_n = v$.

1.3 Introductory Aircraft Pitch Example

For example, consider a simple aircraft pitch-control problem. Suppose we want to control the aircraft pitch angle θ . A simplified nonlinear pitch model could have the simple companion form

$$\begin{aligned} \dot{\theta} &= q \\ \dot{q} &= b(x) + a(x)\delta_e \end{aligned}$$

where q denotes pitch rate, δ_e denotes elevator deflection, $b(x)$ represents the natural pitching dynamics, and $a(x)$ represents elevator control effectiveness. Choosing $y = \theta$ gives $\dot{y} = \dot{\theta} = q = b(x) + a(x)\delta_e$. Introduce the desired pitch angular acceleration as the virtual input v and impose $\ddot{\theta} = v = b(x) + a(x)\delta_e$. Solving for the physical actuator input gives

$$\delta_e = a^{-1}(x)[v - b(x)]$$

The dynamics have now been inverted. The virtual input v is not a physical actuator signal; it specifies the desired pitch angular acceleration. For example, it may request an acceleration of 1.25 rad/s^2 , after which the aircraft model determines the elevator deflection required to realize that acceleration.

Substituting $u = a^{-1}(x)[v - b(x)]$ into the original dynamics gives

$$\frac{d}{dt} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} x_2 \\ x_3 \\ \dots \\ x_n \\ b(x) + a(x)(a^{-1}(x)[v - b(x)]) \end{bmatrix} = \begin{bmatrix} x_2 \\ x_3 \\ \dots \\ x_{n-1} \\ v \end{bmatrix}$$

Consequently, the nonlinear functions $a(x)$ and $b(x)$ are cancelled and the input-output dynamics become a chain of integrators. The difficulty is that a general nonlinear system

does not normally appear in this form. A typical system may instead contain complicated nonlinear coupling between all its states, such as

$$\begin{aligned}\dot{x}_1 &= \sin(x_2) + (x_2 + 1)x_3 \\ \dot{x}_2 &= x_1^5 + x_3 \\ \dot{x}_3 &= x_1^2 + u \\ y &= \cos(x_1)\end{aligned}$$

Although the input u appears explicitly only in $\dot{x}_3$, the measured output is $y = \cos(x_1)$. The relation between u and y is therefore not immediately in controller-companion form. The nonlinear dynamics are reorganized into coordinates based on the output and its derivatives until the control input appears explicitly; the resulting input-output relation can then be inverted.

1.4 Control-Affine Nonlinear Systems

The SISO nonlinear system considered throughout the derivation is written in control-affine form as

$$\dot{x} = f(x) + g(x)u, \quad y = h(x)$$

where $x \in \mathbb{R}^n$, $f(x) \in \mathbb{R}^n$, $g(x) \in \mathbb{R}^n$, while u and y are scalar. A system is control-affine when the input enters linearly even though its coefficients can depend nonlinearly on the state. The system may therefore be highly nonlinear in x while retaining the structure $\dot{x} = f(x) + g(x)u$. This structure is important because NDI ultimately solves algebraically for u .

1.5 Lie Derivatives

The tool used to reveal the input-output structure is the Lie derivative. For a scalar function $h(x)$, the first-order Lie derivative of $h(x)$ along the vector field $f(x)$ is the directional derivative

$$L_f h(x) = \nabla h(x) f(x) = \frac{\partial h(x)}{\partial x} f(x) = \sum_{i=1}^n \frac{\partial h(x)}{\partial x_i} f_i(x)$$

where

$$\nabla h(x) = \frac{\partial h(x)}{\partial x} = \left[\frac{\partial h(x)}{\partial x_1} \quad \frac{\partial h(x)}{\partial x_2} \quad \dots \quad \frac{\partial h(x)}{\partial x_n} \right]$$

The Lie derivative gives the instantaneous rate of change induced by the vector field $f(x)$. When $y = h(x)$ is the measured output $h(x)$, the quantity $L_f h(x)$ describes the contribution of the drift dynamics $f(x)$ to the output derivative.

Lie derivatives can be applied repeatedly. The zeroth-order derivative is simply $L_f^0 h(x)$, the first is $L_f h(x) = \nabla h(x) f(x)$, and higher-order derivatives are defined recursively by

$$L_f^k h(x) = L_f [L_f^{k-1} h(x)]$$

A second vector field such as $g(x)$ can also be used, giving quantities such as

$$L_g L_f^{k-1} h(x) = \nabla [L_f^{k-1} h(x)] g(x)$$

$L_f h(x)$ is the derivative of the output along the drift vector field $f(x)$. The quantity $\frac{\partial h(x)}{\partial x}$ appearing here is the output gradient. Likewise, $L_g L_f h(x)$ measures the contribution of the input vector field $g(x)$ to the next derivative in the output chain. More generally, $L_g L_f^{k-1} h(x)$ determines whether the input appears in the relevant higher-order output derivative associated with index $(k + 1)$.

1.6 Relative Degree

The relative degree formalizes the point at which the control input first appears in the output-derivative sequence.

$$\dot{y} = \frac{\partial h(x)}{\partial x} \dot{x} = \frac{\partial h(x)}{\partial x} [f(x) + g(x)u] = L_f h(x) + L_g h(x)u$$

If $L_g h(x) \neq 0$, the input appears in the first derivative of the output. If $L_g h(x) = 0$, differentiation must continue. The relative degree r is the number of output differentiations required before the input appears explicitly. Around a point x_0 , this is expressed by

$$L_g h(x) = L_g L_f h(x) = \dots = L_g L_f^{r-2} h(x) = 0$$

while $L_g L_f^{r-1} h(x) \neq 0$.

Thus, for relative degree $r = 1$, the input u appears in $\dot{y}$, the first output derivative. For relative degree $r = 2$, the input u first appears in $\ddot{y}$, the second output derivative. In general, the relative degree cannot exceed the system order n .

1.7 Nonlinear Coordinate Transformation

This derivative structure suggests a nonlinear change of coordinates. Define

$$\phi_1(x) = h(x)$$

$$\phi_2(x) = L_f h(x)$$

...

$$\phi_r(x) = L_f^{r-1} h(x)$$

These coordinates correspond to the output and its successive derivatives before the input appears. If $r < n$, they do not yet span the full n -dimensional state space. The original state has n components but only r external coordinates have been introduced. The transformation is therefore completed by $n - r$ additional functions $\phi_{r+1}(x), \dots, \phi_n(x)$, chosen so that

$$L_g \phi_i(x) = 0, \quad r + 1 \leq i \leq n$$

The additional coordinates parameterize the internal dynamics that remain outside the input-output derivative chain. The coordinates $\phi_1(x), \dots, \phi_{r-1}(x)$ represent the output-derivative chain before direct input appearance, $\phi_r(x)$ is the final external coordinate at which the input enters, and $\phi_{r+1}(x), \dots, \phi_n(x)$ complete the transformation with internal coordinates. The complete nonlinear state transformation is

$$z = \Phi(x) = \begin{bmatrix} \phi_1(x) \\ \phi_2(x) \\ \vdots \\ \phi_r(x) \\ \phi_{r+1}(x) \\ \vdots \\ \phi_n(x) \end{bmatrix}$$

$$\underline{z} = \underline{\Phi}(\underline{x}) = \begin{bmatrix} \phi_1(\underline{x}) \\ \phi_2(\underline{x}) \\ \vdots \\ \phi_r(\underline{x}) \\ \phi_{r+1}(\underline{x}) \\ \vdots \\ \phi_n(\underline{x}) \end{bmatrix} \quad (38)$$

No explicit relation with input
 Single equation that contains the input
 internal dynamics, no relation to input

with :
 $\phi_1(\underline{x}) = h(\underline{x})$
 $\phi_2(\underline{x}) = L_f h(\underline{x})$
 $\vdots = \vdots$
 $\phi_r(\underline{x}) = L_f^{r-1} h(\underline{x})$

Locally, the transformation is taken to be invertible so that $x = \Phi^{-1}(z)$.

For the first coordinate z_1 , $\dot{z}_1 = \dot{\phi}_1(x) = \frac{\partial \phi_1(x)}{\partial x} [f(x) + g(x)u] = L_f h(x) + L_g h(x)u$.

Up to the final external coordinate $z_r = \phi_r(x)$, the coordinates $z_i = \phi_i(x)$ do not contain the input directly, consistent with $L_g h(x) = 0$. The derivative chain therefore satisfies $\dot{z}_1 = \dot{\phi}_1(x) = L_f h(x) = z_2$ and then $\dot{z}_2 = z_3$, continuing through $z_{r-1} = z_r$. The final coordinate z_r is the point at which the input becomes explicit:

$$\dot{z}_r = L_f^r h(x) + L_g L_f^{r-1} h(x)u$$

After expressing the original state as $x = \Phi^{-1}(z)$, we can say,

$$\dot{z}_r = L_f^r h(\Phi^{-1}(z)) + L_g L_f^{r-1} h(\Phi^{-1}(z))u = b(z) + a(z)u$$

where $b(z) = L_f^r h(\Phi^{-1}(z))$ and $a(z) = L_g L_f^{r-1} h(\Phi^{-1}(z))$. The transformed external dynamics therefore have the controller-companion structure required for NDI.

For each additional coordinate $z_i = \phi_i(x)$, $r + 1 \leq i \leq n$,

$$\dot{z}_i = L_f \phi_i(x) + L_g \phi_i(x)u$$

Because the internal coordinates are chosen such that $L_g \phi_i(x) = 0$, their dynamics can be written as $\dot{z}_i = q_i(z)$, $q_i(z) = L_f \phi_i(\Phi^{-1}(z))$.

The complete transformed system is therefore

$$\begin{aligned} \dot{z}_1 &= z_2, \\ \dot{z}_2 &= z_3, \\ &\vdots \\ \dot{z}_{r-1} &= z_r, \\ \dot{z}_r &= b(z) + a(z)u, \\ \dot{z}_{r+1} &= q_{r+1}(z), \\ &\vdots \\ \dot{z}_n &= q_n(z) \end{aligned}$$

1.8 External and Internal Dynamics

The first r equations form the external dynamics associated with the measured output, while the last $n - r$ equations are the internal dynamics.

1.9 SISO NDI Control Law

NDI is applied to the single external-dynamics equation that contains the control input:

$$\dot{z}_r = b(z) + a(z)u$$

Substitution of the inversion law gives

$$u = a^{-1}(z)[v - b(z)],$$

$$\dot{z}_r = b(z) + a(z)a^{-1}(z)[v - b(z)] = v$$

The input-output subsystem therefore becomes

$$\begin{aligned}\dot{z}_1 &= z_2, \\ \dot{z}_2 &= z_3, \\ &\vdots \\ \dot{z}_r &= v, \\ y &= h(x) = z_1\end{aligned}$$

With $\dot{y} = z_2, \ddot{y} = z_3, \dots, y^{(r)} = v$, NDI converts the nonlinear input-output dynamics into a linear chain of integrators. This statement concerns only the input-output subsystem. If $r < n$, internal dynamics remain and must also be stable. A virtual input v may yield good tracking, such as $y \rightarrow y_{ref}$, while the internal states still diverge. The internal dynamics can depend nonlinearly on the external/virtual variables, including v ; they must therefore be analysed separately.

1.10 Extension to MIMO Systems

The same ideas extend naturally to MIMO systems. Consider

$$\dot{x} = f(x) + G(x)u, \quad y = h(x)$$

where $x \in \mathbb{R}^n, f(x) \in \mathbb{R}^n, u \in \mathbb{R}^m, y \in \mathbb{R}^m, G \in \mathbb{R}^{n \times m}$.

The input matrix is written in terms of its individual input vector fields, $G(x) = [g_1(x) \ g_2(x) \ \dots \ g_m(x)]$.

For the square direct-inversion formulation considered here, the number of controlled outputs equals the number of inputs. Each column $g_j(x)$ describes how the corresponding input u_j acts on the state dynamics.

Unlike SISO systems, a MIMO system possesses an individual relative degree for each output, $\{r_1, r_2, \dots, r_m\}$.

For output y_i , define its coordinate chain as

$$\begin{aligned}\phi_1^i(x) &= h_i(x) \\ \phi_2^i(x) &= L_f h_i(x) \\ &\dots \\ \phi_{r_i}^i(x) &= L_f^{r_i-1} h_i(x)\end{aligned}$$

The total relative degree is $r = r_1 + r_2 + \dots + r_m$.

If $r < n$, then $n - r$ additional internal coordinates are required. The complete set of output derivative chains occupies r dimensions of the state transformation; the remaining dimensions represent internal dynamics rather than separate full-dimensional transformations for each output.

Within the chain belonging to output y_i ,

$$\begin{aligned} \dot{\phi}_1^i(\underline{x}) &= \phi_2^i(\underline{x}) \\ &\vdots \\ \dot{\phi}_{r_i-1}^i(\underline{x}) &= \phi_{r_i}^i(\underline{x}) \end{aligned} \quad \boxed{G(\underline{x}) = [g_1 \quad g_2 \quad \cdots \quad g_m]} \quad (52)$$

$$\dot{\phi}_{r_i}^i(\underline{x}) = L_{f_i}^{r_i} h_i(\underline{x}) + \sum_{j=1}^m L_{g_j} L_{f_i}^{r_i-1} h_i(\underline{x}) u_j$$

Single equation that contains the input for output i for relative degree r_i .

The final equation is the MIMO analogue of the SISO expression $\dot{z}_r = L_f^r h(x) + L_g L_f^{r-1} h(x)u$. The essential difference is that the highest relevant derivative of each output can depend on multiple inputs, producing a matrix-valued control effectiveness.

Collecting the final equation for each output $y_1, \dots, y_m$, each with its own relative-degree chain $r_1, \dots, r_m$, gives

$$\begin{aligned} \dot{\phi}_{r_1}^1(\underline{x}) &= L_{f_1}^{r_1} h_1(\underline{x}) + \sum_{j=1}^m L_{g_j} L_{f_1}^{r_1-1} h_1(\underline{x}) u_j \\ \dot{\phi}_{r_2}^2(\underline{x}) &= L_{f_2}^{r_2} h_2(\underline{x}) + \sum_{j=1}^m L_{g_j} L_{f_2}^{r_2-1} h_2(\underline{x}) u_j \\ &\vdots \\ \dot{\phi}_{r_m}^m(\underline{x}) &= L_{f_m}^{r_m} h_m(\underline{x}) + \sum_{j=1}^m L_{g_j} L_{f_m}^{r_m-1} h_m(\underline{x}) u_j \end{aligned}$$

In total vector form,

$$v(x) = b(x) + A(x)u$$

$$\underline{v}(\underline{x}) = \begin{bmatrix} \dot{\phi}_{r_1}^1 \\ \dot{\phi}_{r_2}^2 \\ \vdots \\ \dot{\phi}_{r_m}^m \end{bmatrix}, \quad \underline{b}(\underline{x}) = \begin{bmatrix} L_{f_1}^{r_1} h_1(\underline{x}) \\ L_{f_2}^{r_2} h_2(\underline{x}) \\ \vdots \\ L_{f_m}^{r_m} h_m(\underline{x}) \end{bmatrix}$$

Note: g_i is related to input i , h_j to output j .

$$A(\underline{x}) = \begin{bmatrix} L_{g_1} L_{f_1}^{r_1-1} h_1(\underline{x}) & L_{g_2} L_{f_1}^{r_1-1} h_1(\underline{x}) & \cdots & L_{g_m} L_{f_1}^{r_1-1} h_1(\underline{x}) \\ L_{g_1} L_{f_2}^{r_2-1} h_2(\underline{x}) & L_{g_2} L_{f_2}^{r_2-1} h_2(\underline{x}) & \cdots & L_{g_m} L_{f_2}^{r_2-1} h_2(\underline{x}) \\ \vdots & \vdots & \ddots & \vdots \\ L_{g_1} L_{f_m}^{r_m-1} h_m(\underline{x}) & L_{g_2} L_{f_m}^{r_m-1} h_m(\underline{x}) & \cdots & L_{g_m} L_{f_m}^{r_m-1} h_m(\underline{x}) \end{bmatrix} \quad (56)$$

inputs 1 to m $\longrightarrow$ outputs 1 to m

The matrix $A(x)$ is the decoupling matrix. Its rows correspond to outputs and its columns to inputs. The element $A_{ij} = L_{g_j} L_{f_i}^{r_i-1} h_i(x)$ quantifies how input u_j affects the relevant highest derivative of output y_i . The number of differentiations required before this term appears is the relative degree r_i , not the value of the matrix element itself.

If $A(x)$ is invertible, the MIMO NDI law is

$$u = A^{-1}(x)[v(x) - b(x)]$$

where $y_i^{(r_i)} = v_i$.

The nonlinear input-output system has therefore been linearized and the channels have been decoupled: v_i directly commands the highest derivative in the chain associated with

output i . If the total relative degree equals n , these chains occupy the complete state dimension and no internal dynamics remain. If the total relative degree is smaller than n , an additional internal subsystem remains.

1.11 Worked MIMO NDI Example

Consider the following MIMO example:

$$\dot{x} = f(x) + G(x)u, \quad y = h(x)$$

where $y_1 = \cos(x_1)$, $y_2 = \sin(x_3)$.

$$\underline{f}(\underline{x}) = \begin{bmatrix} \sin x_2 + (x_2 + 1)x_3 \\ x_1^5 + x_3 \\ x_1^2 \end{bmatrix}, \quad G(\underline{x}) = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\underline{h}(\underline{x}) = \begin{bmatrix} \cos x_1 \\ \sin x_3 \end{bmatrix}, \quad \underline{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

Consider first the relation between output y_1 and input u_1 .

$$\begin{aligned} z_1^1 &= \phi_1^1(\underline{x}) = L_f^0 h_1(\underline{x}) = h_1(\underline{x}) = \cos x_1 \\ z_2^1 &= \phi_2^1(\underline{x}) = L_f^1 h_1(\underline{x}) = \nabla h_1(\underline{x}) \underline{f}(\underline{x}) = \frac{\partial h_1(\underline{x})}{\partial \underline{x}} \underline{f}(\underline{x}) = \frac{\partial \cos x_1}{\partial \underline{x}} \underline{f}(\underline{x}) = \quad (62a) \\ &= \begin{bmatrix} -\sin x_1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \sin x_2 + (x_2 + 1)x_3 \\ x_1^5 + x_3 \\ x_1^2 \end{bmatrix} = -\sin x_1 [\sin x_2 + (x_2 + 1)x_3] \end{aligned}$$

To determine the relative degree of y_1 , identify the first derivative for which $\sum_{j=1}^m L_{g_j} L_f^{r_1-1} h_1(x) u_j$ is nonzero. For $r_1 = 1$, $\sum_{j=1}^m L_{g_j} L_f^{r_1-1} h_1(x) u_j = 0$; for $r_1 = 2$, $\sum_{j=1}^m L_{g_j} L_f^{r_1-1} h_1(x) u_j \neq 0$. Hence the first output has relative degree $r_1 = 2$. The corresponding relations for y_1 and u_1 are therefore

$$\begin{aligned} \dot{z}_1^1 &= z_2^1 \\ \dot{z}_2^1 &= L_f^2 h_1(x) + \sum_{j=1}^m L_{g_j} L_f h_1(x) u_j = b_1(x) + a_1(x)u = \dots \end{aligned}$$

Repeating the same procedure for output y_2 and input u_2

$$z_1^2 = \phi_1^2(\underline{x}) = L_f^0 h_2(\underline{x}) = h_2(\underline{x}) = \sin x_3$$

gives

Since $r_1 = 2$ and the total relative degree satisfies $r_1 + r_2 = 3$, it follows that $r_2 = 1$. The corresponding relations for y_2 and u_2 are

$$\dot{z}_1^2 = L_f h_2(x) + \sum_{j=1}^m L_{g_j} L_f^0 h_1(x) u_j = b_2(x) + a_2(x)u = \dots$$

The total relative degree is therefore 3, and no internal dynamics remain.

In vector form,

$$v(x) = b(x) + A(x)u$$

Thus,

$$u = A^{-1}(x)[v(x) - b(x)]$$

is the MIMO NDI control law.

2. Aircraft Flight Dynamics for NDI

2.1 Rigid-Body Rotational Dynamics

The rotational dynamics form the basis of the subsequent aircraft-control derivations. In component form, the rigid-body Euler equations relate the applied moments L, M, N to the angular accelerations $\dot{p}, \dot{q}, \dot{r}$, the inertia properties, and nonlinear products of angular rates.

$$\begin{aligned} L &= I_x \dot{p} - (I_y - I_z)qr - I_{xz}(\dot{r} + pq) \\ M &= I_y \dot{q} - (I_z - I_x)rp - I_{zx}(r^2 - p^2) \\ N &= I_z \dot{r} - (I_x - I_y)pq - I_{zx}(\dot{p} - qr) \end{aligned}$$

These equations relate the applied moments to the rigid-body rotational motion. For NDI, the most useful representation is the compact matrix-vector form

$$\begin{aligned} J \dot{\omega} &= -\omega \times J\omega + M, \\ \dot{\omega} &= J^{-1}(M - \omega \times J\omega) \end{aligned} \quad \begin{aligned} L &= \frac{1}{2} \rho V^2 S b C_l \\ M &= \frac{1}{2} \rho V^2 S \bar{c} C_m \\ N &= \frac{1}{2} \rho V^2 S b C_n \end{aligned}$$

$$\begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} I_x & 0 & -I_{xz} \\ 0 & I_y & 0 \\ -I_{zx} & 0 & I_z \end{bmatrix}^{-1} \begin{bmatrix} L \\ M \\ N \end{bmatrix} - \begin{bmatrix} I_x & 0 & -I_{xz} \\ 0 & I_y & 0 \\ -I_{zx} & 0 & I_z \end{bmatrix}^{-1} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times \begin{bmatrix} I_x & 0 & -I_{xz} \\ 0 & I_y & 0 \\ -I_{zx} & 0 & I_z \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

2.2 Aerodynamic Moment Model

The aerodynamic moment can be written as

$$M_a = \frac{1}{2} \rho V^2 S \begin{bmatrix} b C_l \\ \bar{c} C_m \\ b C_n \end{bmatrix}$$

The aerodynamic moment coefficients can be represented by polynomial models. Typical expressions are

$$\begin{aligned} C_l &= C_{l_0} + C_{l_\beta} \beta + C_{l_p} \frac{pb}{2V} + C_{l_r} \frac{rb}{2V} + C_{l_{\delta_a}} \delta_a + C_{l_{\delta_r}} \delta_r + \dots \\ C_m &= C_{m_0} + C_{m_\alpha} \alpha + C_{m_q} \frac{q\bar{c}}{V} + C_{m_{\delta_e}} \delta_e + \dots \\ C_n &= C_{n_0} + C_{n_\beta} \beta + C_{n_p} \frac{pb}{2V} + C_{n_r} \frac{rb}{2V} + C_{n_{\delta_a}} \delta_a + C_{n_{\delta_r}} \delta_r + \dots \end{aligned}$$

The coefficients contain two conceptually different contributions: state-dependent aerodynamic moments, such as those associated with angle of attack, sideslip, and angular rate, and control-dependent moments generated by the control-surface deflections.

2.3 Aircraft Attitude Kinematics

With Euler attitude angles $\theta = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}$, the attitude rates are related to the body angular rates through

$$\dot{\theta} = N(\theta)\omega$$

$$\dot{\phi} = p + \sin \phi \tan \theta q + \cos \phi \tan \theta r$$

$$\dot{\theta} = \cos \phi q - \sin \phi r$$

$$\dot{\psi} = \frac{\sin \phi}{\cos \theta} q + \frac{\cos \phi}{\cos \theta} r$$

Body angular rates are not, in general, equal to Euler-angle rates. This distinction becomes essential when moving from angular-rate control to attitude control.

3. Aircraft Angular-Rate Control with NDI

3.1 Control Objective and Rate Dynamics

For angular-rate control, choose the controlled state $x = \omega = \begin{bmatrix} p \\ q \\ r \end{bmatrix}$, with nonlinear dynamics

$$\dot{x} = \dot{\omega} = J^{-1}(M_a - \omega \times J\omega)$$

When only aerodynamic moments are considered in the total moment,

$$M_a = \frac{1}{2}\rho V^2 S \begin{bmatrix} bC_l \\ \bar{c}C_m \\ bC_n \end{bmatrix}. \text{ For the NDI derivation, the aerodynamic model is reorganized into}$$

state-dependent and control-dependent contributions. Although the true aerodynamic coefficients may be nonlinear and uncertain, a simplified decomposition is used here:

$$\begin{bmatrix} C_l \\ C_m \\ C_n \end{bmatrix} = C_a(x) + C_\delta u,$$

where $C_a(x)$ collects state-dependent terms involving α, β, p, q, r and the zero-offset coefficients, while C_δ contains the control-effectiveness derivatives. For the actuator

ordering $u = \begin{bmatrix} \delta_a \\ \delta_e \\ \delta_r \end{bmatrix}$, the matrix is approximately

$$C_\delta = \begin{bmatrix} C_{l\delta_a} & 0 & C_{l\delta_r} \\ 0 & C_{m\delta_e} & 0 \\ C_{n\delta_a} & 0 & C_{l\delta_r} \end{bmatrix}$$

The off-diagonal lateral-directional terms are important: an aileron can contribute to yawing moment, and a rudder can contribute to rolling moment. The MIMO inversion therefore accounts naturally for cross-coupling.

Substitution into the rigid-body rotational dynamics gives a system of the form

$$\dot{\omega} = J^{-1} \left[\frac{1}{2}\rho V^2 S D C_a(x) - \omega \times J\omega \right] + \frac{J^{-1}}{2}\rho V^2 S D C_\delta u$$

$$|\dot{x} = f(x) + G(x)u|$$

3.2 Relative Degree of Angular-Rate Control

Consider first the relation between the first controlled output and the control inputs. The first output coordinate is

$$z_1^1 = \phi_1^1(x) = h_1(x) = p$$

For the first output p , differentiate once and examine whether the control input appears.

$$\dot{z}_1^1 = L_f p + \sum_{j=1}^3 L_{g_j} p u_j, \quad \sum_{j=1}^3 L_{g_j} p u_j \neq 0$$

Because the control-surface deflections already influence $\dot{p}$, $r_1 = 1$, the input appears after one differentiation, giving

$$\dot{z}_1^1 = b_1(x) + a_1(x)u$$

Repeating the same procedure for the second and third outputs gives

$$\dot{z}_1^2 = b_2(x) + a_2(x)u$$

$$\dot{z}_1^3 = b_3(x) + a_3(x)u$$

The total relative degree is therefore 3, so no internal dynamics remain in this reduced rate-control model. Collecting the final equation from each output chain gives

$$\begin{bmatrix} \dot{z}_1^1 \\ \dot{z}_1^2 \\ \dot{z}_1^3 \end{bmatrix} = \begin{bmatrix} L_f h_1(x) \\ L_f h_2(x) \\ L_f h_3(x) \end{bmatrix} + \begin{bmatrix} L_{g_1} h_1 & L_{g_2} h_1 & L_{g_3} h_1 \\ L_{g_1} h_2 & L_{g_2} h_2 & L_{g_3} h_2 \\ L_{g_1} h_3 & L_{g_2} h_3 & L_{g_3} h_3 \end{bmatrix} \begin{bmatrix} \delta_a \\ \delta_e \\ \delta_r \end{bmatrix}.$$

3.3 NDI Rate-Control Law

Introduce the virtual input $v = \begin{bmatrix} v_p \\ v_q \\ v_r \end{bmatrix} = \begin{bmatrix} z_1^1 \\ z_1^2 \\ z_1^3 \end{bmatrix} = b(x) + A(x)u$. Using $f(x) = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix}$ and the

corresponding Lie-derivative relations $L_f h_1(x) = \nabla h_1(x)f(x) = f_1$, $L_f h_2(x) = f_2$, $L_f h_3(x) = f_3$, the rate dynamics can be collected in vector form as

$$b(x) = f(x), A(x) = G(x)$$

The aircraft angular-rate dynamics can therefore be written in the generic MIMO NDI form

$$v = f(x) + G(x)u$$

Provided that the rate-control decoupling matrix is nonsingular, the nonlinear dynamics can be inverted:

$$u = G(x)^{-1}(v - f(x))$$

Substitution of the aircraft-specific expressions gives

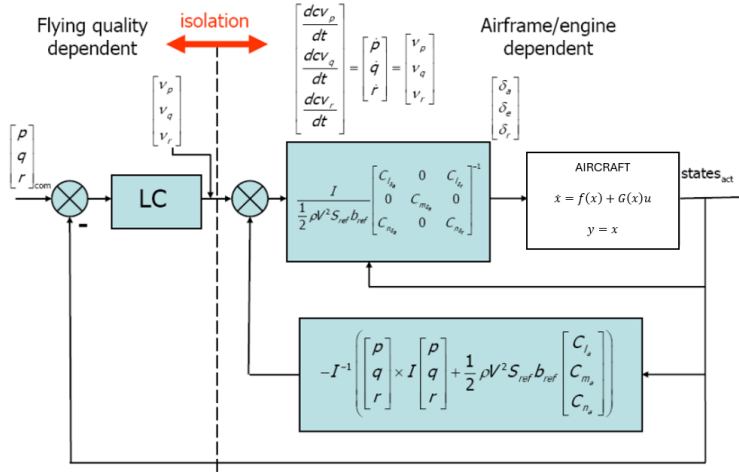
$$\begin{aligned} \begin{bmatrix} \delta_a \\ \delta_e \\ \delta_r \end{bmatrix} &= \begin{bmatrix} bC_{l_{\delta_a}} & 0 & bC_{l_{\delta_r}} \\ 0 & \bar{c}C_{m_{\delta_e}} & 0 \\ bC_{m_{\delta_a}} & 0 & bC_{m_{\delta_r}} \end{bmatrix}^{-1} \left\{ \frac{I}{\frac{1}{2}\rho V^2 S} \begin{bmatrix} \frac{d v_p}{dt} \\ \frac{d v_q}{dt} \\ \frac{d v_r}{dt} \end{bmatrix} + I^{-1} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times I \begin{bmatrix} p \\ q \\ r \end{bmatrix} - \begin{bmatrix} bC_{l_a} \\ \bar{c}C_{m_a} \\ bC_{n_a} \end{bmatrix} \right\} \\ &= \begin{bmatrix} bC_{l_{\delta_a}} & 0 & bC_{l_{\delta_r}} \\ 0 & \bar{c}C_{m_{\delta_e}} & 0 \\ bC_{m_{\delta_a}} & 0 & bC_{m_{\delta_r}} \end{bmatrix}^{-1} \left\{ \frac{I}{\frac{1}{2}\rho V^2 S} \begin{bmatrix} v_p \\ v_q \\ v_r \end{bmatrix} + I^{-1} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times I \begin{bmatrix} p \\ q \\ r \end{bmatrix} - \frac{1}{2}\rho V^2 S \begin{bmatrix} bC_{l_a} \\ \bar{c}C_{m_a} \\ bC_{n_a} \end{bmatrix} \right\} \end{aligned}$$

Substituting this control law back into the rotational dynamics cancels the modeled nonlinear terms and leaves nominally linear integrator dynamics.

3.4 Linear Response Controller

The NDI inverse can be combined with a linear response controller to form a complete angular-rate control system. The inverse itself does not specify the virtual-control components v_p , v_q , and v_r . Instead, the response controller forms the tracking error $e =$

$[p \ q \ r]^T_{com} - [p \ q \ r]^T$ and applies a linear law $v = K(x_{com} - x)$ to generate $v_p = \dot{p}$, $v_q = \dot{q}$, and $v_r = \dot{r}$.



Once the virtual control v has been determined, the NDI law computes the physical control input u . In the block diagram, the expression for u combines the model-

dependent terms and is premultiplied by $\frac{1}{\frac{1}{2}\rho V^2 S_{ref} b_{ref}} \begin{bmatrix} C_{l\delta_a} & 0 & C_{l\delta_r} \\ 0 & C_{m\delta_e} & 0 \\ C_{n\delta_a} & 0 & C_{n\delta_r} \end{bmatrix}^{-1}$. Under exact-

model assumptions, this yields nominally linear integrator dynamics for the controlled states x , which are fed back to the response controller.

3.5 Flying-Quality and Airframe Separation

The controller can be separated into a flying-quality-dependent response controller and an airframe-dependent inverse. The response controller specifies how quickly and smoothly the angular rates should respond, while the inverse determines the actuator motion required by the particular aircraft to realize that response.

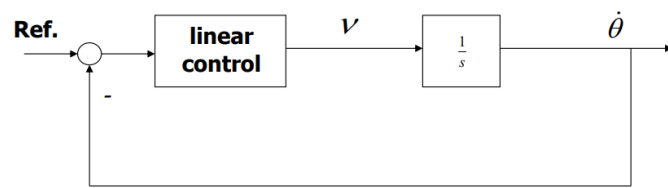
The response controller determines flying-quality characteristics such as bandwidth, damping, and response speed. If aircraft parameters such as $I_x, I_y, I_z, V, C_{l\delta}, \dots$ change, the actuator-to-motion relationship changes as well. With ideal NDI, the plant is represented as $\dot{x} = f(x) + G(x)\delta$ and the inverse uses $\delta = A^{-1}(x)[v - b(x)]$. Model changes therefore appear primarily through $b(x)$ and $A(x)$. If these quantities are estimated online and the inverse is updated accordingly, the nominal relation $\dot{\omega} = v$ can be approximately preserved. An RLS-based identification scheme is one possible mechanism for providing such updates.

After inversion, each angular-rate channel behaves as

$$\dot{p} = v_p, \quad \dot{q} = v_q, \quad \dot{r} = v_r$$

The virtual input therefore sees a simple integrator,

$$\frac{\omega(s)}{v(s)} = \frac{1}{s}$$



The resulting nominal plant seen by the response controller is linearized. The remaining design task is to select the linear response-controller gains.

4. Aircraft Attitude Control with NDI

4.1 Control Objective and State

Attitude control is one dynamic level farther from the physical actuators than angular-rate control: control-surface deflections generate moments, moments generate angular accelerations, angular accelerations change body rates, and body rates change the attitude angles. The corresponding state contains six variables,

Attitude	$\underline{\theta} = [\phi \quad \theta \quad \psi]^T$
Rotational rate	$\underline{\omega} = [p \quad q \quad r]^T$
State vector	$\underline{x} = \begin{bmatrix} \underline{\theta}^T & \underline{\omega}^T \end{bmatrix}^T$
Control input	$\underline{u} = [\delta_e \quad \delta_a \quad \delta_r]^T$
Control variables (output variables)	$\underline{cv} = \underline{y} = \underline{h}(\underline{x}) = [\phi \quad \theta \quad \psi]^T = H\underline{x}$
where	$H = \begin{bmatrix} I_{3 \times 3} & 0_{3 \times 3} \end{bmatrix}$

$$\underline{x} = [\theta \quad \omega]^T = [\phi \quad \theta \quad \psi \quad p \quad q \quad r]^T,$$

while the controlled output is $\underline{y} = h(\underline{x}) = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}$. The output matrix $H = [I_{3 \times 3} \quad 0_{3 \times 3}]$ is chosen

such that $\underline{y} = H\underline{x}$.

4.2 Control-Affine Attitude Model

Combining the attitude kinematics with the rotational dynamics gives the nonlinear system in control-affine form

$$\dot{\underline{x}} = f(\underline{x}) + G(\underline{x})\underline{u}, \quad \underline{y} = h(\underline{x})$$

$$\underline{f}(\underline{x}) = \begin{bmatrix} N(\underline{\theta})\underline{\omega} \\ I^{-1}(\underline{M}_a - \underline{\omega} \times I\underline{\omega}) \end{bmatrix} \quad G(\underline{x}) = \begin{bmatrix} 0_{3 \times 3} \\ \frac{1}{2} \rho V^2 S I^{-1} C_\delta \end{bmatrix}$$

$$\underline{M}_a = \frac{1}{2} \rho V^2 S \begin{bmatrix} bC_{l_a} & \bar{c}C_{m_a} & bC_{n_a} \end{bmatrix}^T$$

$$C_\delta = \begin{bmatrix} bC_{l_{\delta_a}} & 0 & bC_{l_{\delta_r}} \\ 0 & \bar{c}C_{m_{\delta_e}} & 0 \\ bC_{n_{\delta_a}} & 0 & bC_{n_{\delta_r}} \end{bmatrix}$$

$$I = \begin{bmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{yx} & I_{yy} & -I_{yz} \\ -I_{zx} & -I_{zy} & I_{zz} \end{bmatrix}$$

$$N(\underline{\theta}) = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \frac{\sin \phi}{\cos \theta} & \frac{\cos \phi}{\cos \theta} \end{bmatrix}$$

$$\begin{bmatrix} C_{l_a} \\ C_{m_a} \\ C_{n_a} \end{bmatrix} = \begin{bmatrix} C_{l_0} + C_{l_\beta} \beta + C_{l_p} \frac{pb}{2V} + C_{l_r} \frac{rb}{2V} \\ C_{m_0} + C_{m_\alpha} \alpha + C_{m_q} \frac{q\bar{c}}{V} \\ C_{n_0} + C_{n_\beta} \beta + C_{n_p} \frac{pb}{2V} + C_{n_r} \frac{rb}{2V} \end{bmatrix}$$

The upper part of $f(\underline{x})$ contains the attitude kinematics that map body angular rates $\underline{\omega} = [p \quad q \quad r]^T$ into Euler-angle rates for $\underline{\theta} = [\phi \quad \theta \quad \psi]^T$. The lower part of $f(\underline{x})$ contains the rotational dynamics already introduced for angular-rate control.

The upper zero block in $G(\underline{x})$ expresses the central fact that control-surface deflections do not instantaneously change the Euler angles. The actuator vector δ first changes the aerodynamic moments and therefore $\underline{\omega}$; the resulting angular rates $\underline{\omega}$ then change the attitude vector $\phi \quad \theta \quad \psi$. Thus the control surfaces do not directly set $\phi \quad \theta \quad \psi$.

4.3 Relative Degree of Attitude Control

For the aircraft model considered here, each Euler-angle output has relative degree two. This can be verified by differentiating the outputs until the control-surface deflections appear. Consider first the roll-angle output.

$$y_1 = h_1(x) \rightarrow z_1^1 = \phi_1^1(x) = h_1(x) = \phi$$

Its first derivative is

$$\dot{z}_1^1 = L_f h_1(x) + \sum_{j=1}^3 L_{g_j} h_1(x) u_j$$

Because $\sum_{j=1}^3 L_{g_j} h_1(x) u_j = 0$,

$$\dot{z}_1^1 = L_f h_1(x) = z_2^1$$

Using the kinematic relation, define $z_2^1 = L_f h_1(x) = L_f \phi = \dot{\phi} = p + q \sin \phi \tan \theta + r \cos \phi \tan \theta$.

$$\dot{z}_2^1 = L_f^2 h_1(x) + \sum_{j=1}^3 L_{g_j} L_f h_1(x) u_j$$

The control inputs now enter through the rotational accelerations $\dot{p}, \dot{q}, \dot{r}$, so

$$\sum_{j=1}^3 L_{g_j} L_f h_1(x) u_j \neq 0$$

Thus $r_1 = 2$. The second derivative can therefore be written as

$$\dot{z}_2^1 = b_1(x) + a_1(x)u$$

Repeating the derivation for the second and third outputs gives

$$\dot{z}_2^2 = b_2(x) + a_2(x)u$$

$$\dot{z}_2^3 = b_3(x) + a_3(x)u$$

The total relative degree is therefore 6. Because this equals the six-state model dimension, no internal dynamics remain under the stated model assumptions. The final output equations can be collected as

$$\begin{bmatrix} \dot{z}_2^1 \\ \dot{z}_2^2 \\ \dot{z}_2^3 \end{bmatrix} = \begin{bmatrix} L_f^2 h_1(x) \\ L_f^2 h_2(x) \\ L_f^2 h_3(x) \end{bmatrix} + \begin{bmatrix} L_{g_1} L_f h_1 & L_{g_2} L_f h_1 & L_{g_3} L_f h_1 \\ L_{g_1} L_f h_2 & L_{g_2} L_f h_2 & L_{g_3} L_f h_2 \\ L_{g_1} L_f h_3 & L_{g_2} L_f h_3 & L_{g_3} L_f h_3 \end{bmatrix} u$$

Since $\begin{bmatrix} \dot{z}_2^1 \\ \dot{z}_2^2 \\ \dot{z}_2^3 \end{bmatrix} = \begin{bmatrix} \ddot{\phi} \\ \ddot{\theta} \\ \ddot{\psi} \end{bmatrix}$, we can write the standard MIMO NDI form as

$$\ddot{y} = b(x) + A(x)u$$

4.4 Attitude NDI Control Law

NDI proceeds as in the angular-rate case. Introduce the virtual input

$$\mathbf{v} = \begin{bmatrix} v_\phi \\ v_\theta \\ v_\psi \end{bmatrix}$$

Impose $\ddot{\mathbf{y}} = \mathbf{v}$. Since $\mathbf{v} = \mathbf{b}(\mathbf{x}) + \mathbf{A}(\mathbf{x})\mathbf{u}$, the physical control input becomes

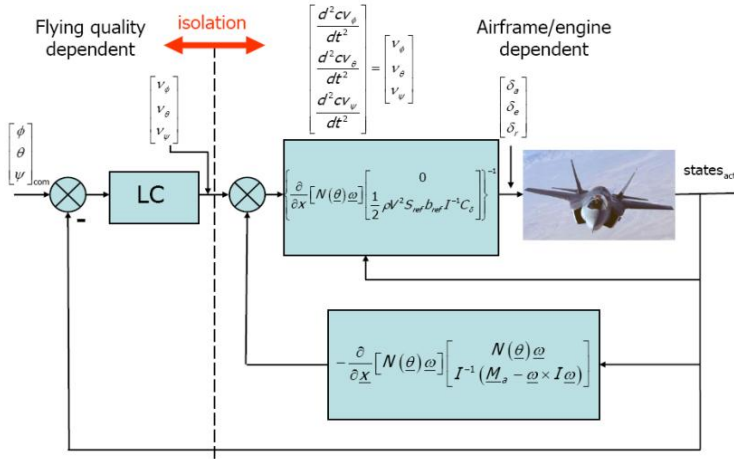
$$\mathbf{u} = \mathbf{A}^{-1}(\mathbf{x})[\mathbf{v} - \mathbf{b}(\mathbf{x})]$$

Evaluating the required Lie derivatives gives

$$\mathbf{M}(\underline{\mathbf{x}}) = \frac{\partial}{\partial \underline{\mathbf{x}}} \left[\mathbf{N}(\underline{\boldsymbol{\theta}}) \underline{\boldsymbol{\omega}} \right] \begin{bmatrix} 0 \\ \frac{1}{2} \rho V^2 S I^{-1} \mathbf{C}_\delta \end{bmatrix}$$

$$\mathbf{l}(\underline{\mathbf{x}}) = \frac{\partial}{\partial \underline{\mathbf{x}}} \left[\mathbf{N}(\underline{\boldsymbol{\theta}}) \underline{\boldsymbol{\omega}} \right] \begin{bmatrix} \mathbf{N}(\underline{\boldsymbol{\theta}}) \underline{\boldsymbol{\omega}} \\ I^{-1} (\underline{\mathbf{M}}_a - \underline{\boldsymbol{\omega}} \times \mathbf{J} \underline{\boldsymbol{\omega}}) \end{bmatrix}$$

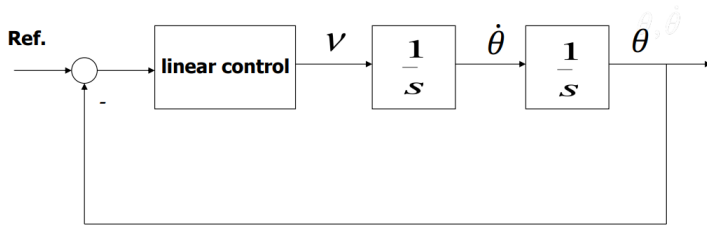
The term $\mathbf{l}(\mathbf{x})$ represents the nonlinear attitude acceleration generated by the current motion, aerodynamics, and inertial coupling. The matrix $\mathbf{M}(\mathbf{x})$ maps control-surface deflections to the second derivative of attitude, and $\mathbf{M}^{-1}(\mathbf{x})$ converts the difference between desired and natural acceleration into actuator commands.



Aircraft attitude control with NDI

The resulting attitude-control architecture separates the response design from the aircraft inverse. The linear response controller generates v_ϕ, v_θ, v_ψ from the desired attitude dynamics, while the NDI block uses the airframe-dependent aerodynamic and rigid-body model to compute the actuator deflections.

After cancellation of the modeled nonlinearities, the nominal attitude dynamics reduce to $\ddot{\boldsymbol{\theta}} = \mathbf{v}$. Each channel therefore behaves as a double integrator, $\frac{\theta(s)}{v(s)} = \frac{1}{s^2}$.



4.5 Attitude Response Controller

For a reference trajectory θ_{ref} , a PD-type virtual-control law can be chosen as

$$v = \ddot{\theta}_{ref} - K_p(\theta - \theta_{ref}) - K_d(\dot{\theta} - \dot{\theta}_{ref})$$

The gain K_p provides restoring action on attitude error, while K_d provides damping through attitude-rate error. The reference-acceleration term supplies feedforward. Because the nominal NDI dynamics are linearized, these gains can be selected using classical linear-control methods.

4.6 Rate Control versus Attitude Control

The distinction between angular-rate and attitude NDI is summarized by relative degree. For angular-rate control, the outputs are p, q, r , and the actuator deflections appear after one differentiation:

$$p \rightarrow \dot{p} \rightarrow u, \quad q \rightarrow \dot{q} \rightarrow u, \quad r \rightarrow \dot{r} \rightarrow u$$

Each angular-rate channel has relative degree one, giving $r_{total} = 1 + 1 + 1 = 3$. This equals the dimension of the three-state angular-rate model.

For attitude control, the outputs are ϕ, θ, ψ . Their first derivatives depend on body angular rates but not directly on the actuators; the actuators appear only in the second derivatives:

$$\phi \rightarrow \dot{\phi} \rightarrow \ddot{\phi} \rightarrow u,$$

$$\theta \rightarrow \dot{\theta} \rightarrow \ddot{\theta} \rightarrow u,$$

$$\psi \rightarrow \dot{\psi} \rightarrow \ddot{\psi} \rightarrow u.$$

Each attitude channel therefore has relative degree two, giving $r_{total} = 2 + 2 + 2 = 6$.

The attitude-control state consists of three attitude angles and three angular rates, also giving six states. Consequently, under the assumptions made in this model, the sum of the relative degrees equals the state dimension and there are no remaining internal dynamics.

5. Time-Scale-Separated NDI and Sideslip Control

5.1 Motivation for Time-Scale Separation

Time-scale separation is useful when a system contains dynamics with clearly different response speeds. A local measure of control effectiveness is $\partial y_j / \partial u_i$, the sensitivity of an output to a control input. An elevator deflection δ_e affects pitch angular acceleration first, then pitch rate, and only later pitch angle. Consequently, the control effectiveness associated with δ_e and q is much stronger than that associated with δ_e and θ .

$$\delta_e \rightarrow \dot{q} \rightarrow q \rightarrow \dot{\theta} \rightarrow \theta$$

5.2 Why Direct Slow-Variable Inversion Is Undesirable

Consider vertical position Z controlled through elevator deflection δ_e . The actuator-to-position response contains several intermediate dynamic stages:

$$\delta_e \rightarrow M \rightarrow \dot{q} \rightarrow q \rightarrow \theta, \alpha \rightarrow L \rightarrow \gamma \rightarrow \dot{Z} \rightarrow Z$$

Suppose the aircraft is 100 m below the desired altitude, so $Z_{ref} - Z = 100$.

A naive direct law such as $\delta_e = K_Z(Z_{ref} - Z)$ would translate the large position error directly into elevator deflection. For the output q with input δ_e , the relative degree is one in the simplified rate model, whereas for the output Z with input δ_e the relative degree is much higher. Several dynamic stages and integrations separate actuator motion from vertical position, while fast states such as p, q, r can respond substantially earlier. For example, the elevator produces the rapid intermediate response $\delta_e \rightarrow \dot{q}$ before the position changes appreciably. A single direct inversion from position to actuator would therefore require a high-order derivative chain before the physical input u appears and would not exploit the natural separation between slow and fast dynamics.

Instead, decompose the control problem into layers, each with its own controller. The position controller first generates a desired flight-path angle from the position error.

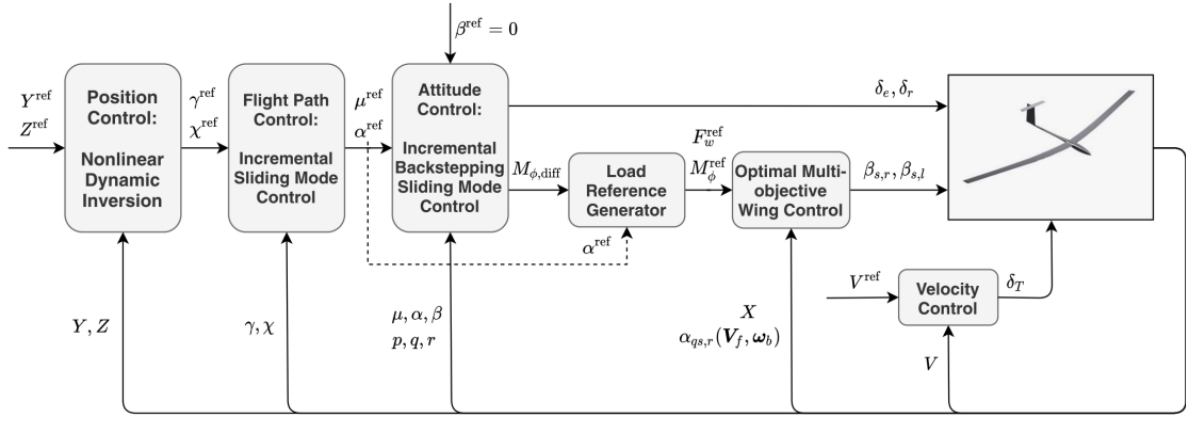
$$Z_{ref} - Z \rightarrow \gamma_{ref} = 5^\circ$$

The next layer converts the flight-path-angle error into a desired angle of attack.

$$\gamma_{ref} - \gamma \rightarrow \alpha_{ref} = 6^\circ$$

The cascade eventually produces the elevator command δ_e . Thus, rather than using the direct mapping $Z_{ref} \rightarrow \delta_e$, the controller uses a layered mapping such as $Z_{ref} \rightarrow \gamma_{ref} \rightarrow \alpha_{ref} \rightarrow q_{ref} \rightarrow \delta_e$.

The accompanying block diagram illustrates the resulting cascaded flight-control architecture.



5.3 Cascaded NDI Architecture

Time-scale-separated NDI decomposes the nonlinear dynamics into a slow outer kinematic system and a fast inner rotational system. NDI is applied separately to each layer. The outer inversion generates an angular-rate reference rather than a physical actuator command, and the inner inversion converts that reference into control-surface deflections.

5.4 Fast Inner Rate Loop

Consider first the fast inner angular-rate loop. Define

$$\begin{aligned}\underline{x}_1 &= \underline{\omega} \\ \underline{y}_1 &= \underline{h}_1(\underline{x}_1) = \underline{\omega} \\ \underline{u}_1 &= [\delta_a \quad \delta_e \quad \delta_r]^T\end{aligned}$$

The rotational dynamics can be written in the control-affine form $\dot{\underline{x}}_1 = f_1(\underline{x}_1) + G_1(\underline{x}_1)\underline{u}_1$.

$$\underline{f}_1(\underline{x}_1) = -I^{-1}(\underline{\omega} \times I \underline{\omega}) + I^{-1} \frac{1}{2} \rho V^2 S \begin{bmatrix} bC_{l_a} \\ \bar{c}C_{m_a} \\ bC_{n_a} \end{bmatrix}; \quad G_1(\underline{x}_1) = I^{-1} \frac{1}{2} \rho V^2 S \begin{bmatrix} bC_{l_{\delta_a}} & 0 & bC_{l_{\delta_r}} \\ 0 & \bar{c}C_{m_{\delta_e}} & 0 \\ bC_{n_{\delta_a}} & 0 & bC_{n_{\delta_r}} \end{bmatrix}$$

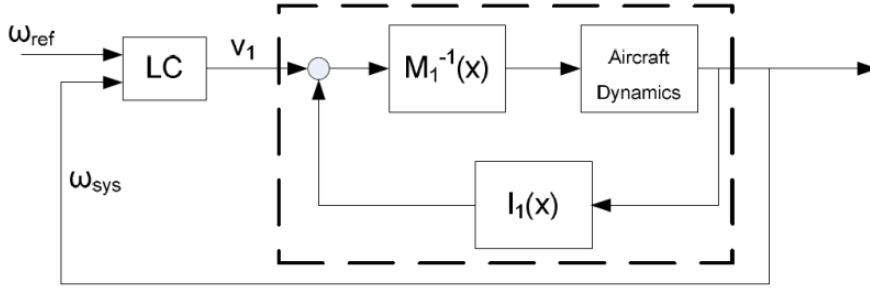
Differentiating the output once gives

$$\begin{aligned}\frac{d\underline{h}_1(\underline{x}_1)}{dt} &= \frac{\partial \underline{h}_1(\underline{x}_1)}{\partial \underline{x}_1} \frac{d\underline{x}_1}{dt} = \frac{\partial \underline{h}_1(\underline{x}_1)}{\partial \underline{x}_1} [\underline{f}_1(\underline{x}_1) + G_1(\underline{x}_1)\underline{u}_1] \\ &= \underline{f}_1(\underline{x}_1) + G_1(\underline{x}_1)\underline{u}_1\end{aligned}$$

This is the standard NDI relation $\dot{\underline{y}}_1 = \underline{l}_1(\underline{x}_1) + \underline{M}_1(\underline{x}_1)\underline{u}_1$, with $\underline{l}_1(\underline{x}_1) = \underline{f}_1(\underline{x}_1)$ and $\underline{M}_1(\underline{x}_1) = \underline{G}_1(\underline{x}_1)$. Setting the desired output derivative equal to the virtual control $\underline{v}_1$ gives $\dot{\underline{y}}_1 = \underline{l}_1(\underline{x}_1) + \underline{M}_1(\underline{x}_1)\underline{u}_1$, so the dynamic-inversion law is

$$\begin{aligned}
\underline{u}_1(\underline{x}_1) &= \underline{M}_1^{-1}(\underline{x}_1) [\underline{v}_1 - \underline{l}_1(\underline{x}_1)] = \underline{G}_1^{-1}(\underline{x}_1) [\underline{v}_1 - \underline{f}_1(\underline{x}_1)] \\
&= \frac{I}{\frac{1}{2}\rho V^2 S} \begin{bmatrix} C_{l_{\delta_a}} & 0 & C_{l_{\delta_r}} \\ 0 & C_{m_{\delta_e}} & 0 \\ C_{n_{\delta_a}} & 0 & C_{n_{\delta_r}} \end{bmatrix}^{-1} \left\{ \underline{v}_1 + I^{-1} \underline{\omega} \times I \underline{\omega} - I^{-1} \frac{1}{2} \rho V^2 S \begin{bmatrix} C_{l_a} \\ C_{m_a} \\ C_{n_a} \end{bmatrix} \right\}
\end{aligned}$$

Substitution into the nonlinear rotational dynamics cancels the modeled terms and leaves the nominal input-output relation $\dot{y}_1 = \dot{\omega} = v_1$.



Given the angular-rate reference ω_{ref} , the inner response controller generates the virtual angular-acceleration command v_1 . The inverse $M_1^{-1}(x)$ then maps the remaining acceleration requirement into the actuator command, while the model term $l_1(x)$ is cancelled using state feedback.

5.5 Slow Outer Kinematic Loop

The slow outer loop treats the inner-loop-controlled aircraft as a new system. Define

$$\begin{aligned}
\underline{x}_2 &= \underline{\theta} \\
\underline{y}_2 &= \underline{h}_2(\underline{x}_2) = \underline{\theta} \\
\underline{u}_2 &= \underline{\omega}
\end{aligned}$$

From the perspective of the slow outer kinematic system, the body angular rates $u_2 = \omega$ are treated as virtual control inputs. Using the rotational kinematics $\dot{\theta} = N(\theta)\omega$, the outer system can be written as $\dot{x}_2 = f_2(x_2) + G_2(x_2)u_2$, with

$$\begin{aligned}
\underline{f}_2(\underline{x}_2) &= \underline{0} \\
\underline{G}_2(\underline{x}_2) &= \underline{N}(\underline{\theta}) = \underline{\Omega}_{\theta\omega}
\end{aligned}$$

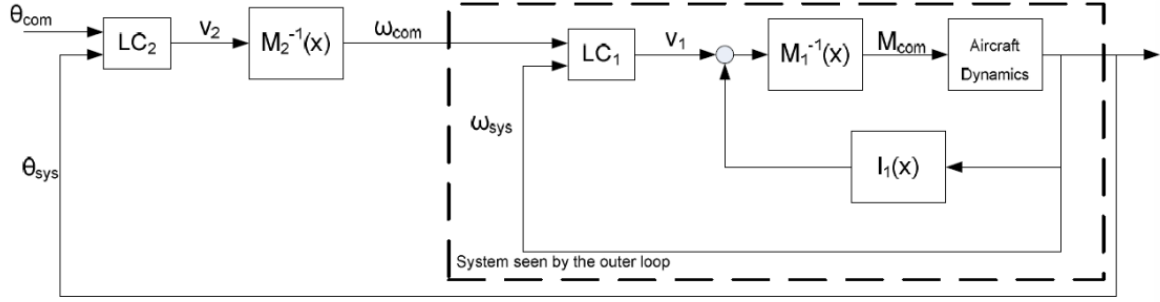
Differentiating the outer-loop output gives

$$\begin{aligned}
\frac{d\underline{h}_2(\underline{x}_2)}{dt} &= \frac{\partial \underline{h}_2(\underline{x}_2)}{\partial \underline{x}_2} \frac{d\underline{x}_2}{dt} = \frac{\partial \underline{h}_2(\underline{x}_2)}{\partial \underline{x}_2} [\underline{f}_2(\underline{x}_2) + \underline{G}_2(\underline{x}_2)\underline{u}_2] \\
&= \underline{f}_2(\underline{x}_2) + \underline{G}_2(\underline{x}_2)\underline{u}_2 = \underline{N}(\underline{\theta})\underline{\omega} = \underline{\Omega}_{\theta\omega}\underline{\omega} = \dot{\underline{\theta}}
\end{aligned}$$

The outer virtual control $v_2 = \dot{\theta}$ can therefore be inverted to obtain the angular-rate command

$$\begin{aligned}\underline{\omega}_{ref} = \underline{u}_2(\underline{x}_2) &= \underline{M}_2^{-1}(\underline{x}_2) [\underline{v}_2 - \underline{l}_2(\underline{x}_2)] = \underline{G}_2^{-1}(\underline{x}_2) [\underline{v}_2 - \underline{f}_2(\underline{x}_2)] \\ &= \underline{\Omega}_{\theta\omega}^{-1} \underline{v}_2 = \underline{N}^{-1}(\underline{\theta}) \underline{v}_2\end{aligned}$$

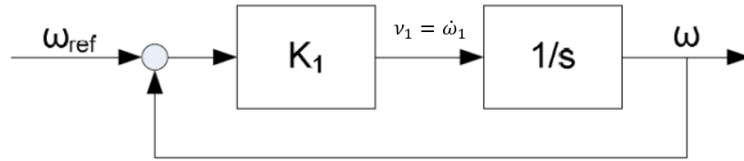
The outer loop therefore converts the attitude command θ_{ref} into a required body-rate reference ω_{ref} , while the fast inner loop converts ω_{ref} into physical actuator commands δ_e . The result is a cascaded two-loop architecture.



The outer response controller LC2 generates v_2 , the kinematic inverse generates ω_{ref} or ω_{com} , and this body-rate command enters the inner NDI system consisting of LC1, the dynamic inverse, and the aircraft dynamics.

5.6 Combined Closed-Loop Dynamics

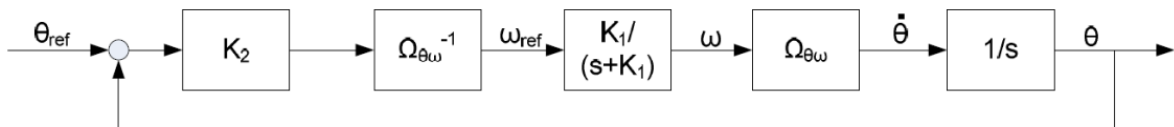
For insight, consider decoupled scalar channels with proportional response controllers. Under ideal NDI, the inner loop can be approximated by the following linear model:



For a single decoupled channel, the corresponding closed-loop transfer function is

$$H_1(s) = \frac{\frac{K_1}{s}}{1 + \frac{K_1}{s}} = \frac{K_1}{s + K_1}$$

The inner-loop response is first order, with its speed determined by K_1 . The outer loop can be analysed similarly; under the same scalar approximation, the combined outer/inner dynamics are



The same scalar approximation can be compared with the full block architecture. The corresponding channel-wise transfer function becomes

$$\begin{aligned}
H(s) &= \frac{\frac{K_2 \Omega_{\theta\omega}^{-1} H_1(s) \Omega_{\theta\omega}}{s}}{1 + \frac{K_2 \Omega_{\theta\omega}^{-1} H_1(s) \Omega_{\theta\omega}}{s}} = \frac{K_2 \Omega_{\theta\omega}^{-1} H_1(s) \Omega_{\theta\omega}}{s + K_2 \Omega_{\theta\omega}^{-1} H_1(s) \Omega_{\theta\omega}} = \frac{K_2 \Omega_{\theta\omega}^{-1} \frac{K_1}{s + K_1} \Omega_{\theta\omega}}{s + K_2 \Omega_{\theta\omega}^{-1} \frac{K_1}{s + K_1} \Omega_{\theta\omega}} \\
&= \frac{K_2 \Omega_{\theta\omega}^{-1} K_1 \Omega_{\theta\omega}}{s(s + K_1) + K_2 \Omega_{\theta\omega}^{-1} K_1 \Omega_{\theta\omega}} = \frac{K_2 K_1}{s^2 + K_1 s + K_2 K_1} \quad (120)
\end{aligned}$$

This transfer-function expression is a single-channel, decoupled approximation and does not represent the full MIMO coupling of the nonlinear controller.

5.7 Sideslip Control and Kinematics

The preceding architecture controls the Euler-angle vector $\theta = [\phi, \theta, \psi]^T$. For the flight-control design considered here, yaw angle ψ is replaced by sideslip angle β , giving the outer-loop controlled variables $[\phi, \theta, \beta]^T$. The objective is therefore to derive the kinematic relation between $\dot{\beta}$ and the body rates (p,q,r).

Define the measurable body specific-force components

$$A_x = \frac{X}{m}, A_y = \frac{Y}{m}, A_z = \frac{Z}{m}$$

The resulting sideslip kinematic equation is

$$\begin{aligned}
\dot{\beta} &= \frac{1}{\sqrt{u^2 + w^2}} \left[\frac{-uv}{V^2} (A_x - g \sin \theta) + \left(1 - \frac{v^2}{V^2} \right) (A_y + g \sin \phi \cos \theta) - \frac{vw}{V^2} (A_z + g \cos \phi \cos \theta) \right] + \\
&\quad + \begin{bmatrix} \frac{w}{\sqrt{u^2 + w^2}} & 0 & \frac{-u}{\sqrt{u^2 + w^2}} \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}
\end{aligned}$$

Combining the roll and pitch kinematics with the sideslip relation gives the outer-loop kinematic model for the selected controlled variables,

$$\begin{aligned}
\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\beta} \end{bmatrix} &= \begin{bmatrix} 0 \\ 0 \\ \frac{1}{\sqrt{u^2 + w^2}} \left[\frac{-uv}{V^2} (A_x - g \sin \theta) + \left(1 - \frac{v^2}{V^2} \right) (A_y + g \sin \phi \cos \theta) - \frac{vw}{V^2} (A_z + g \cos \phi \cos \theta) \right] \end{bmatrix} + \\
&\quad + \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ \frac{w}{\sqrt{u^2 + w^2}} & 0 & \frac{-u}{\sqrt{u^2 + w^2}} \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}
\end{aligned}$$

5.8 Complete Time-Scale-Separated Controller

The combined kinematic relation forms the basis of the complete time-scale-separated NDI controller. First, references for ϕ , θ , and β are processed by the outer response controllers to generate the desired virtual angle-rate vector

$$\begin{bmatrix} v_\phi \\ v_\theta \\ v_\beta \end{bmatrix} = \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\beta} \end{bmatrix} = K_{P_1} \left(\begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{ref} - \begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{act} \right) + K_{I_1} \int \left(\begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{ref} - \begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{act} \right) dt + K_{D_1} \frac{d}{dt} \left(\begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{ref} - \begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{act} \right) + \frac{d}{dt} \begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{ref}$$

or only proportional controls

$$\begin{bmatrix} v_\phi \\ v_\theta \\ v_\beta \end{bmatrix} = \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\beta} \end{bmatrix} = K_{P_1} \left(\begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{ref} - \begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{act} \right) + \frac{d}{dt} \begin{bmatrix} \phi \\ \theta \\ \beta \end{bmatrix}_{ref}$$

Second, the nonlinear kinematic relation in Eq. (142) is inverted to obtain the desired body-rate vector.

$$\begin{bmatrix} p \\ q \\ r \end{bmatrix}_{ref} = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ \frac{w}{\sqrt{u^2 + w^2}} & 0 & \frac{-u}{\sqrt{u^2 + w^2}} \end{bmatrix}^{-1} \left(\begin{bmatrix} v_\phi \\ v_\theta \\ v_\beta \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ \frac{1}{\sqrt{u^2 + w^2}} \left[\frac{-uv}{V^2} a_x + \left(1 - \frac{v}{V^2} \right) a_y - \frac{vw}{V^2} a_z \right] \right] \right)$$

where

$$\begin{aligned} a_x &= (A_x - g \sin \theta) \\ a_y &= (A_y + g \sin \phi \cos \theta) \\ a_z &= (A_z + g \cos \phi \cos \theta) \end{aligned}$$

The kinematic inversion converts the outer-loop commands $\phi_{ref}, \theta_{ref}, \beta_{ref}$ into body-rate references $p_{ref}, q_{ref}, r_{ref}$. Third, the fast inner response controllers compare these references with the measured rates and generate virtual angular-acceleration commands.

$$\begin{bmatrix} v_p \\ v_q \\ v_r \end{bmatrix} = K_{P_2} \left(\begin{bmatrix} p \\ q \\ r \end{bmatrix}_{ref} - \begin{bmatrix} p \\ q \\ r \end{bmatrix}_{act} \right) + K_{I_2} \int \left(\begin{bmatrix} p \\ q \\ r \end{bmatrix}_{ref} - \begin{bmatrix} p \\ q \\ r \end{bmatrix}_{act} \right) dt + K_{D_2} \frac{d}{dt} \left(\begin{bmatrix} p \\ q \\ r \end{bmatrix}_{ref} - \begin{bmatrix} p \\ q \\ r \end{bmatrix}_{act} \right) + \frac{d}{dt} \begin{bmatrix} p \\ q \\ r \end{bmatrix}_{ref}$$

or only proportional controls:

$$\begin{bmatrix} v_p \\ v_q \\ v_r \end{bmatrix} = K_{P_2} \left(\begin{bmatrix} p \\ q \\ r \end{bmatrix}_{ref} - \begin{bmatrix} p \\ q \\ r \end{bmatrix}_{act} \right) + \frac{d}{dt} \begin{bmatrix} p \\ q \\ r \end{bmatrix}_{ref}$$

Fourth, the rotational dynamics are inverted to obtain the physical control-surface deflections. In compact form, this is the same dynamic inversion derived for the inner loop:

$$\begin{bmatrix} \delta_a \\ \delta_e \\ \delta_r \end{bmatrix} = \begin{bmatrix} bC_{l_{\delta_a}} & 0 & bC_{l_{\delta_r}} \\ 0 & \bar{c}C_{m_{\delta_e}} & 0 \\ bC_{n_{\delta_a}} & 0 & bC_{n_{\delta_r}} \end{bmatrix}^{-1} \left\{ \frac{I}{\frac{1}{2}\rho V^2 S} \left(\begin{bmatrix} v_p \\ v_q \\ v_r \end{bmatrix} + I^{-1} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times I \begin{bmatrix} p \\ q \\ r \end{bmatrix} - \frac{1}{2}\rho V^2 S \begin{bmatrix} bC_{l_a} \\ \bar{c}C_{m_a} \\ bC_{n_a} \end{bmatrix} \right) \right\}$$

Thus, time-scale-separated attitude control replaces one high-order direct inversion from attitude to actuators by two lower-order inversions connected in cascade: a slow kinematic inversion and a fast rotational dynamic inversion.

NDI theory

- Input to state linearization
- Input-output linearization
- General state coordinate transformation
- Internal dynamics

$$\begin{array}{ll} \dot{x} = f(x) + G(x)u & \xrightarrow{\text{Lie derivatives}} \dot{\eta} = f_0(\eta, \xi) = \frac{\partial \phi}{\partial x} f(x) \Big|_{x=T^{-1}(z)} \\ y = h(x) & \dot{\xi} = A_c \xi + B_c [\alpha(x) + \mathcal{B}(x)u] \\ & y = C_c \xi \end{array}$$

Flight control using NDI

- Attitude control → a direct mapping from theta to u
- Angular control → a direct mapping from omega to u
- Attitude control based on time scale separation → cascaded mapping from theta to u

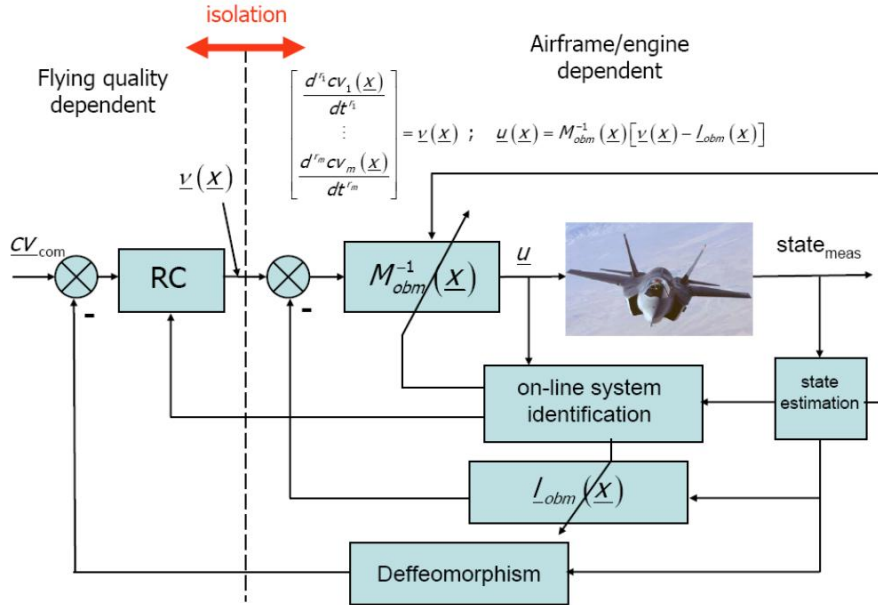
6. Adaptive Nonlinear Dynamic Inversion (ANDI)

6.1 Limitation of Conventional NDI

Adaptive NDI addresses the principal limitation of conventional NDI: the quality of the inversion depends on the accuracy of the model being inverted. Changes in aerodynamics, inertial properties, damage, or an inaccurate nominal database degrade the cancellation and therefore the nominal input-output linearization.

6.2 Adaptive NDI Architecture

The accompanying block diagram shows the conceptual architecture of adaptive NDI.



The left-hand side of the architecture is flying-quality dependent. The commanded controlled variable cv_{com} is compared with the measured or estimated controlled variable, and the response controller generates the virtual control v . The airframe-dependent part then uses dynamic inversion to convert the desired derivative information in v into actuator commands u . State estimation supplies the states required by the controller, while online system identification updates the model used by the inverse. The diffeomorphism represents a nonlinear coordinate transformation from the physical state x to the selected controlled variables cv ; in general, the controlled variables are nonlinear functions of the state.

To make the role of the aircraft model explicit, reconsider the angular-rate NDI control law derived previously:

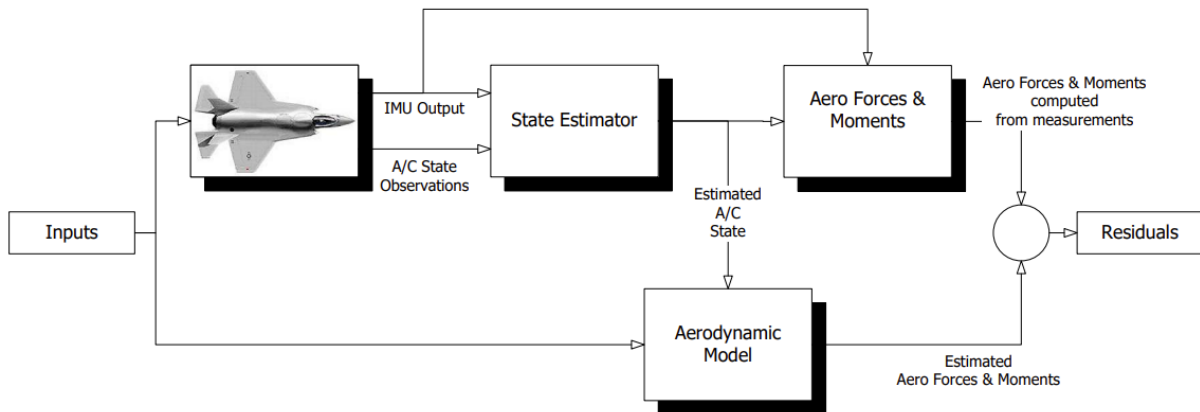
$$\begin{bmatrix} \delta_a \\ \delta_e \\ \delta_r \end{bmatrix} = \begin{bmatrix} bC_{l_{\delta_a}} & 0 & bC_{l_{\delta_r}} \\ 0 & \bar{c}C_{m_{\delta_e}} & 0 \\ bC_{m_{\delta_a}} & 0 & bC_{m_{\delta_r}} \end{bmatrix}^{-1} \left\{ \frac{I}{\frac{1}{2}\rho V^2 S} \begin{bmatrix} \frac{d\dot{c}v_p}{dt} \\ \frac{d\dot{c}v_q}{dt} \\ \frac{d\dot{c}v_r}{dt} \end{bmatrix} + I^{-1} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times I \begin{bmatrix} p \\ q \\ r \end{bmatrix} - \begin{bmatrix} bC_{l_a} \\ \bar{c}C_{m_a} \\ bC_{n_a} \end{bmatrix} \right\}$$

$$= \begin{bmatrix} bC_{l_{\delta_a}} & 0 & bC_{l_{\delta_r}} \\ 0 & \bar{c}C_{m_{\delta_e}} & 0 \\ bC_{m_{\delta_a}} & 0 & bC_{m_{\delta_r}} \end{bmatrix}^{-1} \left\{ \frac{I}{\frac{1}{2}\rho V^2 S} \begin{bmatrix} v_p \\ v_q \\ v_r \end{bmatrix} + I^{-1} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times I \begin{bmatrix} p \\ q \\ r \end{bmatrix} - \frac{1}{2}\rho V^2 S \begin{bmatrix} bC_{l_a} \\ \bar{c}C_{m_a} \\ bC_{n_a} \end{bmatrix} \right\}$$

Question: what happens if there is a **mismatch** between the real system and these models?

6.3 Two-Step Method for Online Identification

A model mismatch occurs when the real aircraft differs from the model used by the inverse. Adaptive NDI addresses this by updating the aircraft model online. The Two-Step Method (TSM) separates the joint state-and-parameter estimation problem into state estimation followed by parameter estimation. For detailed estimation theory, see the companion AE4302 System Identification notes.



The state estimator provides an estimate of the aircraft state, which is used to reconstruct aerodynamic force and moment coefficients. An aerodynamic model is then evaluated with current parameter estimates and compared with these reconstructed coefficients. The resulting residual drives the parameter update so that the model can follow changes in the observed aircraft dynamics.

$$C_X = C_{X_0} + C_{X_\alpha} \alpha + C_{X_{\alpha^2}} \alpha^2 + C_{X_q} \frac{q\bar{c}}{V} + C_{X_{\delta_e}} \delta_e + C_{X_{T_c}} T_c$$

$$C_Z = C_{Z_0} + C_{Z_\alpha} \alpha + C_{Z_q} \frac{q\bar{c}}{V} + C_{Z_{\delta_e}} \delta_e + C_{Z_{T_c}} T_c$$

$$C_m = C_{m_0} + C_{m_\alpha} \alpha + C_{m_q} \frac{q\bar{c}}{V} + C_{m_{\delta_e}} \delta_e + C_{m_{T_c}} T_c$$

$$C_Y = C_{Y_0} + C_{Y_\beta} \beta + C_{Y_p} \frac{pb}{2V} + C_{Y_r} \frac{rb}{2V} + C_{Y_{\delta_a}} \delta_a + C_{Y_{\delta_r}} \delta_r + \boxed{C_{Y_\alpha} \alpha + C_{Y_q} \frac{q\bar{c}}{V}}$$

$$C_l = C_{l_0} + C_{l_\beta} \beta + C_{l_p} \frac{pb}{2V} + C_{l_r} \frac{rb}{2V} + C_{l_{\delta_a}} \delta_a + C_{l_{\delta_r}} \delta_r + \boxed{C_{l_\alpha} \alpha + C_{l_q} \frac{q\bar{c}}{V}}$$

$$C_n = C_{n_0} + C_{n_\beta} \beta + C_{n_p} \frac{pb}{2V} + C_{n_r} \frac{rb}{2V} + C_{n_{\delta_a}} \delta_a + C_{n_{\delta_r}} \delta_r + \boxed{C_{n_\alpha} \alpha + C_{n_q} \frac{q\bar{c}}{V}}$$

6.4 Reconstruction of Aerodynamic Forces

To identify aerodynamic coefficients, the aerodynamic forces and moments must first be reconstructed from measurements. In the body-fixed frame, the aerodynamic forces satisfy

$$\begin{aligned} X &= \frac{1}{2} \rho V^2 S C_X & C_X &= \frac{2m}{\rho V^2 S} A_x & A_x &= \frac{X}{m} = \frac{1}{2m} \rho V^2 S C_X \\ Y &= \frac{1}{2} \rho V^2 S C_Y & C_Y &= \frac{2m}{\rho V^2 S} A_y & A_y &= \frac{Y}{m} = \frac{1}{2m} \rho V^2 S C_Y \\ Z &= \frac{1}{2} \rho V^2 S C_Z & C_Z &= \frac{2m}{\rho V^2 S} A_z & A_z &= \frac{Z}{m} = \frac{1}{2m} \rho V^2 S C_Z \end{aligned}$$

Measured accelerations can therefore be converted into reconstructed aerodynamic force coefficients for use in the parameter-estimation step.

6.5 Reconstruction of Aerodynamic Moments

The same reconstruction is performed for the moments. The angular rates p, q, r are measured by rate gyros, and angular acceleration is obtained from their time derivative $\dot{p}, \dot{q}, \dot{r}$. The rigid-body rotational equations and known inertia properties can then be used to reconstruct the aerodynamic body-axis moments:

$$\begin{aligned} L &= I_x \dot{p} - (I_y - I_z) q r - I_{xz} (\dot{r} + p q) & L &= \frac{1}{2} \rho V^2 S b C_l \\ M &= I_y \dot{q} - (I_z - I_x) r p - I_{zx} (r^2 - p^2) & M &= \frac{1}{2} \rho V^2 S \bar{c} C_m \\ N &= I_z \dot{r} - (I_x - I_y) p q - I_{zx} (\dot{p} - q r) & N &= \frac{1}{2} \rho V^2 S b C_n \end{aligned}$$

The reconstructed aerodynamic moment coefficients C_l, C_m, C_n can then be used in the parameter-estimation step.

6.6 Online Parameter Estimation

Once state estimates are available, the aerodynamic model becomes a standard regression problem $y_k = h_k^T p$. For example,

$$y_k = C_X(k)$$

$$\underline{h}_k^T = \begin{bmatrix} 1 & \alpha(k) & \alpha^2(k) & \frac{q(k)\bar{c}}{V(k)} & \delta_e(k) & T_c(k) \end{bmatrix}$$

$$\underline{p} = \begin{bmatrix} C_{X_0} & C_{X_\alpha} & C_{X_{\alpha^2}} & C_{X_q} & C_{X_{\delta_e}} & C_{X_{T_c}} \end{bmatrix}^T$$

Adaptive recursive least squares (RLS) can be used to update the aerodynamic coefficients continuously during flight. See the companion AE4302 notes for the detailed RLS theory.

7. Incremental Nonlinear Dynamic Inversion (INDI)

7.1 Motivation for INDI

Incremental Nonlinear Dynamic Inversion (INDI) reduces the dependence on the complete nonlinear model $f(x)$. Instead of computing an absolute actuator command u from the full model, INDI computes a small actuator increment Δu relative to the currently applied command u_0 . The updated command is then $u = u_0 + \Delta u$. The simplest INDI formulation therefore requires substantially less model information than conventional NDI.

7.2 Local Incremental Model

Consider the general nonlinear, not necessarily control-affine, system $\dot{x} = f(x, u)$.

Around the current operating point (x_0, u_0) , a first-order Taylor expansion gives

$$\begin{aligned} \dot{x} &\approx f(x_0, u_0) + \left. \frac{\partial f(x, u)}{\partial x} \right|_{x=x_0, u=u_0} (x - x_0) + \left. \frac{\partial f(x, u)}{\partial u} \right|_{x=x_0, u=u_0} (u - u_0) \\ &\approx \dot{x}_0 + F(x_0, u_0)(x - x_0) + G(x_0, u_0)(u - u_0) \end{aligned} \quad (205)$$

measurement of the state derivative!

The incremental formulation starts from the measured current state derivative $\dot{x}_0$ rather than predicting the complete nonlinear dynamics at the present operating point.

7.3 High-Sampling-Rate Approximation

For a sufficiently high control-update frequency, the state changes only slightly over one sampling interval. Under this high-rate approximation, $\Delta x = x - x_0$ is small, so the term $F\Delta x$ may be neglected relative to the direct effect of the new control increment, giving

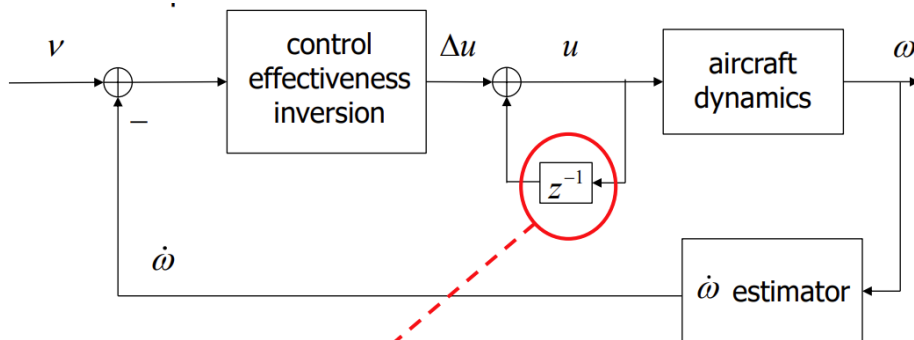
$$\dot{x} \approx \dot{x}_0 + G(x_0, u_0)\Delta u$$

7.4 INDI Control Law

Setting the desired derivative equal to the virtual control $v = \dot{x}$ gives

$$\Delta u = G^{-1}(x_0, u_0)(v - \dot{x}_0)$$

The key advantage is now visible. Conventional NDI uses $u = G^{-1}(x_0, u_0)(v - f(x))$ and therefore requires an explicit model of the nonlinear drift term. INDI instead uses $\Delta u = G^{-1}(x_0, u_0)(v - \dot{x}_0)$, replacing most of the drift-model dependence by a measurement of the aircraft's current response. The main model quantity retained in the basic law is the control-effectiveness matrix G .



7.5 Discrete-Time Implementation

In discrete time, INDI computes an actuator increment Δu rather than an entirely new absolute command.

The updated actuator command is $u_k = u_{k-1} + \Delta u_k$. The unit-delay operator z^{-1} stores the previous sample: z^{-1} applied to the sampled control sequence returns u_{k-1} , as expressed by $z^{-1}u_k = u_{k-1}$. The newly computed increment is added to this stored command. The architecture also feeds back the measured or estimated angular acceleration so that the remaining acceleration requirement is mapped through the inverse control effectiveness into Δu .

7.6 Robustness of NDI versus INDI

To compare robustness, first consider conventional NDI under model uncertainty. Let the nominal onboard model be $\dot{x} = f_0(x) + G_0(x)u$.

The subscript 0 denotes the nominal model. If it were exact, the NDI law $u = G_0^{-1}(\nu - f_0(x))$ would produce $\dot{x} = \nu$. The actual aircraft, however, may satisfy

$$\dot{x} = f_0(x) + \Delta f(x) + [G_0(x) + \Delta G(x)]u$$

Here Δf represents uncertainty in the input-independent dynamics and ΔG represents control-effectiveness uncertainty. Substituting the nominal NDI law into the actual system leaves terms involving both Δf and ΔG . Consequently, the nominally cancelled nonlinear dynamics reappear in closed loop, and the resulting dynamics need not remain linear.

INDI changes this situation because the current derivative is measured. An uncertain incremental system can be written schematically as

$$\dot{x} = \dot{x}_0 + [G_0 + \Delta G]\Delta u$$

The input-independent uncertainty that would have appeared explicitly in $f(x)$ has already influenced the real aircraft and is therefore embedded in the measured $\dot{x}_0$. Applying $\Delta u = G_0^{-1}(\nu - \dot{x}_0)$ leaves primarily terms associated with the control-effectiveness error ΔG . Thus much of the input-independent model uncertainty is removed from the explicit inversion problem by measurement.

The remaining uncertainty is also reduced because INDI operates on small increments. Control-effectiveness errors multiply the incremental command rather than the full control action. As the sampling interval decreases, consecutive state derivatives and control increments become smaller, reducing the influence of control-effectiveness mismatch. The resulting robustness still depends on sufficiently accurate derivative measurements and a sufficiently high update rate.

7.7 Assumptions and Limitations

These assumptions are essential. INDI is model-light rather than model-free: it still requires the control-effectiveness matrix and relies on accurate angular-acceleration information, high sampling rates, and compatible actuator dynamics. If the actuator response is too slow, the previously commanded input may not represent the control effect actually present at the aircraft, weakening the incremental approximation.

8. Adaptive INDI

8.1 Motivation and Complete Incremental Model

Basic INDI neglects $F\Delta x$ because the state increment is assumed small over one sampling interval. This assumption can become questionable for highly maneuverable vehicles or rapidly changing/damaged aircraft. In that case, the complete local incremental model must be retained:

$$\dot{\underline{x}} \simeq \dot{\underline{x}}_0 + F(\underline{x}_0, \underline{u}_0)(\underline{x} - \underline{x}_0) + G(\underline{x}_0, \underline{u}_0)(\underline{u} - \underline{u}_0)$$

The complete local incremental model contains both state and control Jacobians. If F and G are unknown or time-varying, their elements can be estimated online.

8.2 Regression Formulation

Subtracting the current derivative from the expanded model gives

$$\Delta \dot{\underline{x}} \simeq F(\underline{x}_0, \underline{u}_0) \Delta \underline{x} + G(\underline{x}_0, \underline{u}_0) \Delta \underline{u}$$

The relation is linear in the unknown elements of F and G . For the i -th equation,

$$\Delta \dot{x}_{i,k} = \underline{f}_i(\underline{x}_0, \underline{u}_0) \Delta \underline{x}_k + \underline{g}_i(\underline{x}_0, \underline{u}_0) \Delta \underline{u}_k$$

where $\underline{f}_i$ is the i -th row of F and $\underline{g}_i$ is the i -th row of G . Rearranging gives

$$\begin{aligned} \Delta \dot{x}_{i,k} &= \underline{f}_i(\underline{x}_0, \underline{u}_0) \Delta \underline{x}_k + \underline{g}_i(\underline{x}_0, \underline{u}_0) \Delta \underline{u}_k \\ &= \begin{bmatrix} \underline{f}_i(\underline{x}_0, \underline{u}_0) & \underline{g}_i(\underline{x}_0, \underline{u}_0) \end{bmatrix} \begin{bmatrix} \Delta \underline{x}_k \\ \Delta \underline{u}_k \end{bmatrix} \\ &= \begin{bmatrix} \Delta \underline{x}_k^T & \Delta \underline{u}_k^T \end{bmatrix} \begin{bmatrix} \underline{f}_i^T(\underline{x}_0, \underline{u}_0) \\ \underline{g}_i^T(\underline{x}_0, \underline{u}_0) \end{bmatrix} \end{aligned}$$

The identification problem is therefore linear in the unknown local Jacobian elements $\underline{f}_i, \underline{g}_i$. The dependent variable is an increment in state derivative, while the regressors are measured state and input increments.

8.3 Practical Timing Problem

A practical timing issue arises when angular acceleration is obtained by differentiating gyro measurements. The next-sample angular acceleration is not yet available when the current control command is computed. At sufficiently high sampling frequency, the current acceleration estimate can be used as an approximation, so the same high-rate assumption that supports basic INDI also supports online identification of the complete incremental model.

8.4 Windowed Online Identification

For time-varying F and G , a moving-window or piecewise estimation scheme can be used. Rather than allowing the complete flight history to dominate the estimate, only the most recent incremental measurements are retained and stacked:

$$y = H\theta$$

$$\underline{y} = \begin{bmatrix} \Delta \dot{\underline{x}}_{i,k} \\ \Delta \dot{\underline{x}}_{i,k-1} \\ \vdots \\ \Delta \dot{\underline{x}}_{i,k-N} \end{bmatrix}, \quad H = \begin{bmatrix} \Delta \underline{x}_k^T & \Delta \underline{u}_k^T \\ \Delta \underline{x}_{k-1}^T & \Delta \underline{u}_{k-1}^T \\ \vdots & \vdots \\ \Delta \underline{x}_{k-N}^T & \Delta \underline{u}_{k-N}^T \end{bmatrix}, \quad \underline{\theta} = \begin{bmatrix} \underline{f}_i^T(\underline{x}_0, \underline{u}_0) \\ \underline{g}_i^T(\underline{x}_0, \underline{u}_0) \end{bmatrix}$$

Ordinary least squares then gives

$$\hat{\underline{\theta}} = (H^T H)^{-1} H^T \underline{y}$$

As the window advances, old measurements leave the data set and new measurements enter, allowing the identified local model to follow changes in the aircraft dynamics.

9. Backstepping Control Design

9.1 Motivation for Backstepping

Backstepping is a nonlinear control-design method for systems with a hierarchical or lower-triangular structure. The controller is constructed recursively: at each stage, the next state is temporarily treated as a virtual control, a stabilizing target for that state is designed, and the procedure continues until the physical actuator input is reached.

9.2 Lyapunov Stability Background

The stability argument is based on Lyapunov theory. For a nonlinear system with equilibrium at $x = 0$, a Lyapunov function $V(x)$ is a scalar measure of the system's distance or generalized energy relative to the equilibrium. A suitable Lyapunov function is positive definite,

$$V(0) = 0, \quad V(x) > 0, \quad x \neq 0$$

The minimum of a positive-definite Lyapunov function occurs at the desired equilibrium. Stability is assessed by evaluating its derivative along system trajectories:

$$\dot{V} = \frac{\partial V}{\partial x} \dot{x}$$

If $\dot{V} \leq 0$, the Lyapunov function is non-increasing; under the usual Lyapunov conditions, this can establish stability. If the stronger condition $\dot{V} < 0$ holds for all $x \neq 0$, the Lyapunov function decreases away from the equilibrium and the state converges asymptotically to the equilibrium. In controller design, the input is therefore selected so that the derivative

$V(x)$ is negative away from the equilibrium. A Lyapunov function for which the control input can be chosen to enforce the desired decrease is commonly called a control Lyapunov function (CLF).

9.3 Recursive Principle of Backstepping

Backstepping extends this Lyapunov argument recursively. After a virtual control is designed, its tracking error is introduced as a new state-like error coordinate and the Lyapunov function is augmented by a positive term in that error. The next virtual or physical control is then chosen so that the derivative of the augmented Lyapunov function retains the desired sign.

This recursive construction can be interpreted as selective nonlinear dynamic inversion. Conventional NDI generally cancels nonlinear terms to impose simpler dynamics, whereas backstepping only cancels terms when required by the stability design. Naturally stabilizing nonlinearities can therefore be retained and used rather than cancelled.

9.4 Backstepping as Selective NDI

The value of selective cancellation can first be illustrated with the nonlinear first-order system

$$\dot{x} = -x^3 + x + u$$

The objective is to achieve $x(t) \rightarrow 0$ as $t \rightarrow \infty$.

Choose the positive-definite Lyapunov function

$$V(x) = \frac{1}{2}x^2$$

Its derivative along the system trajectories is

$$\dot{V} = \dot{x}x = x(-x^3 + x + u)$$

The control input u can therefore be selected so that $\dot{V} < 0$ for every $x \neq 0$.

A full NDI cancellation would choose $u = x^3 - kx, k > 1$. The controller term x^3 exactly cancels the naturally occurring plant term $-x^3$, leaving

$$\dot{x} = -(k - 1)x$$

and consequently

$$\dot{V} = -(k - 1)x^2 < 0$$

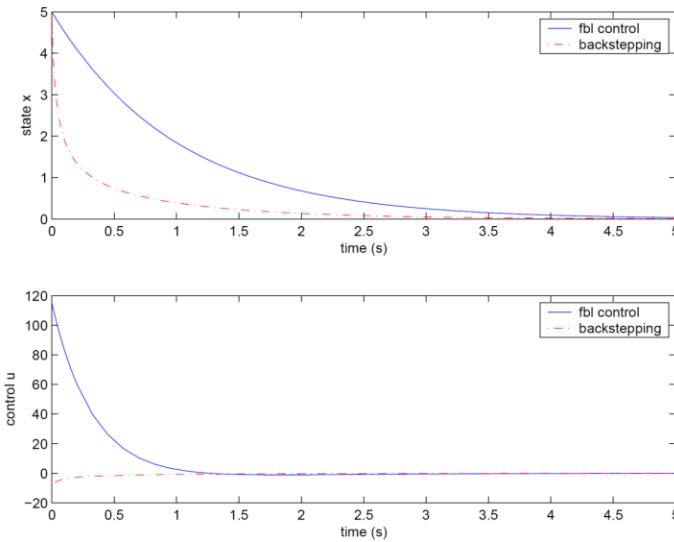
The closed-loop system is asymptotically stable, but full cancellation removes the naturally stabilizing term $-x^3$. Its sign opposes the state: when x is positive, $\dot{x}$ is driven toward the origin; when x is negative, the state is again driven toward the origin. Cancelling this natural damping requires the compensating actuator term $+x^3$, increasing control effort.

A more selective controller is

$$u = -kx, k > 1$$

With the selective controller, $\dot{x} = -x^3 - (k - 1)x$, and hence $\dot{V} = -x^4 - (k - 1)x^2 < 0$

for every $x \neq 0$. Global asymptotic stability is retained while the natural damping term $-x^3$ contributes to stabilization instead of being cancelled. The resulting controller requires less control effort and is less dependent on exact nonlinear cancellation.



The accompanying simulation compares the state x and control input u for the full-cancellation and selective-cancellation controllers using $x(0) = 5$ and $k = 2$. It illustrates that retaining naturally stabilizing nonlinear dynamics can achieve strong convergence with less control effort.

9.5 Two-State Backstepping Example

The recursive nature of backstepping becomes apparent when the physical actuator does not appear in the derivative of the variable to be stabilized. Consider

$$\dot{x} = x^2 - x^3 + \xi, \quad \dot{\xi} = u$$

The objective remains $x \rightarrow 0$. Start with

$$V_1 = \frac{1}{2}x^2$$

which gives $\dot{V}_1 = x(x^2 - x^3 + \xi)$

The difficulty is that the physical input u does not appear in $\dot{V}_1$. Therefore u cannot yet be selected directly to make this expression negative. The state ξ does appear, so backstepping temporarily treats ξ as a virtual input to the x -subsystem.

Suppose ξ could be chosen freely. A stabilizing desired value is

$$\xi_{des} = -x^2 - k_1x = \alpha(x), \quad k_1 > 0$$

If $\xi = \xi_{des}$, then

$$\dot{x} = x^2 - x^3 - x^2 - k_1x = -x^3 - k_1x$$

and therefore

$$\dot{V}_1 = -x^4 - k_1x^2 < 0$$

Thus $\alpha(x)$ is a stabilizing function for the x -subsystem. However, ξ is a state rather than a physical actuator command and cannot be set instantaneously to ξ_{des} . Define the virtual-control tracking error $z = \xi - \xi_{des}$.

The design objective is now twofold: drive $x \rightarrow 0$ while also driving $z \rightarrow 0$. The latter condition enforces $\xi \rightarrow \xi_{des}$, allowing the stabilizing virtual command to be realized by the actual state.

Next derive the dynamics of the tracking error z . Differentiating gives $\dot{z} = \dot{\xi} - \dot{\xi}_{des} = u - \dot{\xi}_{des}$. Using the chosen stabilizing function $\xi_{des} = -x^2 - k_1x$, gives $\dot{\xi}_{des} = -(2x + k_1)\dot{x}$.

$$\xi = z + \xi_{des} = z - x^2 - k_1x$$

so that

$$\dot{x} = x^2 - x^3 + \xi = -x^3 - k_1x + z$$

Consequently,

$$\dot{z} = u - (2x + k_1)(k_1x + x^3 - z)$$

The Lyapunov function is now augmented rather than replaced:

$$V_2 = V_1 + \frac{1}{2}z^2 = \frac{1}{2}x^2 + \frac{1}{2}z^2$$

The physical input u is then chosen so that $\dot{V}_2 < 0$.

With this choice, both x and z are driven toward zero.

The recursive augmentation of the Lyapunov function is a defining feature of backstepping. Differentiating the enlarged function gives $\dot{V}_2 = \dot{x}x + z\dot{z}$.

After substitution,

$$\begin{aligned}
\dot{V}_2 &= x(-x^3 - k_1x) + z \left[u - (2x + k_1)(k_1x + x^3 - z) + x \right] \\
&= x(-x^3 - k_1x) + z \left\{ \left[-x + (2x + k_1)(k_1x + x^3 - z) - k_2z \right] - (2x + k_1)(k_1x + x^3 - z) + x \right\} \\
&= -x^4 - k_1x^2 - k_2z^2
\end{aligned}$$

Choose the physical control law

$$u = -x + (2x + k_1)(k_1x + x^3 - z) - k_2z, k_2 > 0$$

With the selected control law, the bracketed terms cancel except for the stabilizing term $-k_2z$. The resulting Lyapunov derivative is negative definite, implying $x \rightarrow 0, z \rightarrow 0$; since $z = \xi - \xi_{des}$, it also follows that $\xi \rightarrow \xi_{des}$.

9.6 Backstepping versus Feedback Linearization

The same system can also be treated by conventional input-output feedback linearization. Starting from

$$\dot{x} = x^2 - x^3 + \xi,$$

Differentiating once gives

$$\ddot{x} = (2x - 3x^2)\dot{x} + \dot{\xi} = (2x - 3x^2)(x^2 - x^3 + \xi) + u$$

The nonlinear term can then be cancelled through an NDI law of the form

$$u = v - (2x - 3x^2)(x^2 - x^3 + \xi)$$

The pseudo-control v then determines the desired second-order dynamics. This approach stabilizes the system but cancels the entire nonlinear structure, including the useful term $-x^3$. Backstepping instead preserves the naturally stabilizing nonlinearity, motivating its interpretation as selective nonlinear dynamic inversion.

9.7 General Integrator Backstepping

The procedure can be generalized systematically as integrator backstepping. Consider

$$\dot{x}_1 = f(x_1) + g(x_1)x_2, \dot{x}_2 = u$$

Suppose x_1 must track a smooth reference $y_r(t)$ whose required derivatives are known and bounded. Define the first tracking error $z_1 = x_1 - y_r$. Then

$$\dot{z}_1 = f(x_1) + g_1(x)z_2 - \dot{y}_r$$

At this stage x_2 is treated as a virtual control for the z_1 -subsystem. Choose the stabilizing function $x_{2,des} = \alpha(x_1, y_r, \dot{y}_r)$ such that the Lyapunov function $V_1(z_1)$ satisfies

$$\dot{V}_1 = \frac{\partial V_1}{\partial z_1} [f(x_1) + g(x_1)\alpha(x_1, y_r, \dot{y}_r) - \dot{y}_r] \leq -W(z_1),$$

where $W(z_1)$ is positive definite. If x_2 exactly followed α , the first subsystem would already have the desired stability property.

Define the discrepancy between the actual state x_2 and the stabilizing value as $z_2 = x_2 - \alpha(x_1, y_r, \dot{y}_r)$.

Accordingly, $x_2 = z_2 + \alpha$. Substituting this relation into the first error dynamics gives $\dot{z}_1 = f(x_1) + g(x_1)(z_2 + \alpha) - \dot{y}_r$.

The derivative of z_2 contains the physical input:

$$\dot{z}_2 = u - \dot{\alpha}$$

Since α is defined as $\alpha(x_1, y_r, \dot{y}_r)$,

$$\dot{\alpha} = \frac{\partial \alpha}{\partial x_1} \dot{x}_1 + \frac{\partial \alpha}{\partial y_r} \dot{y}_r + \frac{\partial \alpha}{\partial \dot{y}_r} \ddot{y}_r$$

Substituting the system dynamics gives

$$\dot{z}_2 = u - \frac{\partial \alpha}{\partial x_1} [f(x_1) + g(x_1)(z_2 + \alpha)] - \frac{\partial \alpha}{\partial y_r} \dot{y}_r - \frac{\partial \alpha}{\partial \dot{y}_r} \ddot{y}_r$$

These derivatives are known design quantities because α is constructed by the controller designer; its partial derivatives can therefore be evaluated analytically.

Augment the Lyapunov function as

$$V_2(z_1, z_2) = V(z_1) + \frac{1}{2} z_2^2$$

Expanding the derivative preserves the first-stage stabilizing term $-W(z_1)$, while the remaining terms are multiplied by z_2 . The physical input can therefore be selected as

$$u = -cz_2 - \frac{\partial V_1}{\partial z_1} g(x_1) + \frac{\partial \alpha}{\partial x_1} [f(x_1) + g(x_1)(z_2 + \alpha)] + \frac{\partial \alpha}{\partial y_r} \dot{y}_r + \frac{\partial \alpha}{\partial \dot{y}_r} \ddot{y}_r$$

where $c > 0$. The resulting Lyapunov inequality is

$$\dot{V}_2 \leq -W(z_1) - cz_2^2$$

Both the original tracking error and the virtual-control tracking error therefore converge. The second design stage extends the first-stage Lyapunov argument rather than replacing it.

9.8 Dynamic Extension and Singular NDI

Dynamic extension can also be used when direct NDI encounters a singular inversion matrix. Consider the following system:

$$\begin{aligned} \dot{x}_1 &= u_1 \\ \dot{x}_2 &= e^{x_1} u_1 + x_3 & y_1 &= x_1 \\ \dot{x}_3 &= u_2 & y_2 &= x_2 \end{aligned} \quad \frac{d}{dt} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 0 \\ x_3 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ e^{x_1} & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

The original resolving matrix is singular, so direct inversion cannot independently determine u_1 and u_2 .

Introduce dynamic extension by defining $u_1 = w_1, u_2 = w_2$.

Differentiating the outputs again gives

$$\frac{d^2}{dt^2} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 0 \\ e^{x_1} u_1^2 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ e^{x_1} & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}.$$

The resulting resolving matrix is nonsingular, so the extended system can be inverted:

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ e^{x_1} & 1 \end{bmatrix}^{-1} \left(\begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} - \begin{bmatrix} 0 \\ e^{x_1} u_1^2 \end{bmatrix} \right).$$

The important consequence is that one of the original inputs is now treated dynamically by the extension. The remaining input-output dynamics can be viewed in terms of u_1 and x_3 , with x_3 serving as a virtual control whose desired behavior is enforced recursively through the extended dynamics.

9.9 Strict-Feedback Systems

The recursive construction becomes particularly systematic for a strict-feedback system,

$$\begin{aligned} \dot{x} &= f(x) + g(x)\xi_1, \\ \dot{\xi}_1 &= f_1(x, \xi_1) + g_1(x, \xi_1)\xi_2, \\ \dot{\xi}_2 &= f_2(x, \xi_1, \xi_2) + g_2(x, \xi_1, \xi_2)\xi_3, \\ &\vdots \\ \dot{\xi}_k &= f_k(x, \xi_1, \dots, \xi_k) + g_k(x, \xi_1, \dots, \xi_k)u. \end{aligned}$$

The characteristic feature is triangular dependence: each equation contains the next state, while the true input u appears only in the final equation. This remains a SISO system: there are multiple states but only one physical input. Backstepping follows the chain recursively from the controlled state toward the actuator.

9.10 Recursive Backstepping Derivation

At the first stage, treat ξ_1 as if it were directly controllable and choose a continuously differentiable stabilizing function

$$\xi_{1,des} = \alpha(x)$$

together with a positive-definite $V(x)$ such that

$$\frac{\partial V}{\partial x} (f(x) + g(x)\alpha(x)) < -W(x)$$

where $W(x)$ is positive definite. This establishes the desired behavior of the x -subsystem under the hypothetical condition $\xi_1 = \alpha(x)$.

Because ξ_1 is a state rather than a physical input, define the tracking error $z_1 = \xi_1 - \alpha(x)$. Then

$$\dot{z}_1 = \dot{\xi}_1 - \alpha(\dot{x}) = f_1(x, \xi_1) + g_1(x, \xi_1)\xi_2 - \frac{\partial \alpha}{\partial x} (f(x) + g(x)[\alpha(x) + z_1])$$

The augmented Lyapunov function is $V_1 = V(x) + \frac{1}{2}z_1^2$.

The next state ξ_2 is treated as the next virtual control. The objective is to design

$$\xi_{2,des} = \alpha_1(x, z_1)$$

such that the derivative of $\dot{V}_1$ is negative definite. Differentiation gives

$$\dot{V}_1 = \dot{V}(x) + z_1 \dot{z}_1 = \frac{\partial V(x)}{\partial x} \{f(x) + g(x)[\alpha(x) + z_1]\} + z_1 \dot{z}_1 < -W(x) + \frac{\partial V(x)}{\partial x} g(x) z_1 + z_1 \dot{z}_1$$

Substituting the $\dot{z}_1$ relation gives

$$\begin{aligned} \dot{V}_1 < -W(x) + \frac{\partial V(x)}{\partial x} g(x) z_1 + \\ + z_1 \left\{ f_1(x, \xi_1) + g_1(x, \xi_1) \xi_2 - \frac{\partial \alpha(x)}{\partial x} \{f(x) + g(x)[\alpha(x) + z_1]\} \right\} \end{aligned}$$

Assuming $g_1(x, \xi_1) \neq 0$, choose the following virtual control:

$$\begin{aligned} \xi_{2,des} &= \alpha_1(x, z_1) \\ &= \frac{1}{g_1(x, \xi_1)} \left\{ -k_1 z_1 - \frac{\partial V(x)}{\partial x} g(x) - f_1(x, \xi_1) + \frac{\partial \alpha(x)}{\partial x} \{f(x) + g(x)[\alpha(x) + z_1]\} \right\} \end{aligned}$$

With this choice, the Lyapunov derivative becomes

$$\dot{V}_1 < -W(x) - k_1 z_1^2,$$

which is negative definite. Each term in the virtual-control expression is chosen to preserve the negative contribution inherited from the preceding Lyapunov stage while adding a new stabilizing term.

At the next stage, introduce the extended state $X_1 = \begin{bmatrix} x \\ \xi_1 \end{bmatrix}$. The first two equations can be written collectively as

$$\dot{X}_1 = F_1(X_1) + G_1(X_1) \xi_2$$

followed by

$$\dot{\xi}_2 = f_2(X_1, \xi_2) + g_2(X_1, \xi_2) \xi_3$$

This has the same structural form as the preceding stage. Define

$$z_2 = \xi_2 - \alpha_1(X_1),$$

Augment the Lyapunov function again as $V_2 = V_1(X_1) + \frac{1}{2} z_2^2$ and design

$$\xi_{3,des} = \alpha_2(X_1, z_2)$$

so that $\dot{V}_2$ is negative definite. The recursion continues until the final state is reached; only at that stage is the physical actuator input designed.

9.11 General Recursive Backstepping Formula

The construction can now be generalized to an arbitrary strict-feedback chain.

$$\begin{aligned}
\dot{x}_1 &= f_1(x_1) + g_1(x_1)x_2, \\
\dot{x}_2 &= f_2(x_1, x_2) + g_2(x_1, x_2)x_3, \\
&\vdots \\
\dot{x}_i &= f_i(x_1, \dots, x_i) + g_i(x_1, \dots, x_i)x_{i+1}, \\
&\vdots \\
\dot{x}_n &= f_n(x_1, \dots, x_n) + g_n(x_1, \dots, x_n)u,
\end{aligned}$$

The objective is to make $y = x_1$ asymptotically track $y_r(t)$. Assume the required derivatives of the reference are known and bounded. Define the first recursive error coordinate by $z_1 = x_1 - y_r = x_1 - \alpha_0$, and the remaining coordinates by

$$z_i = x_i - \alpha_{i-1}, i = 2, \dots, n$$

Each recursive error coordinate measures the mismatch between the current physical state and the virtual control requested by the preceding layer.

In these coordinates, the dynamics take the form

$$\begin{aligned}
\dot{z}_1 &= f_1(x_1) + g_1(x_1)x_2 - \dot{y}_r, \\
\dot{z}_2 &= f_2(x_1, x_2) + g_2(x_1, x_2)x_3 - \dot{\alpha}_1,
\end{aligned}$$

and, for the intermediate stages,

$$\dot{z}_i = f_i(x_1, \dots, x_i) + g_i(x_1, \dots, x_i)x_{i+1} - \dot{\alpha}_{i-1},$$

with the final equation

$$\dot{z}_n = f_n(x_1, \dots, x_n) + g_n(x_1, \dots, x_n)u - \dot{\alpha}_{n-1}.$$

The control Lyapunov functions are constructed recursively as

$$V_i = V_{i-1} + \frac{1}{2}z_i^2$$

For the standard recursive construction, the first virtual control is

$$\alpha_1 = \frac{1}{g_1}(-c_1 z_1 - f_1 + \dot{y}_r)$$

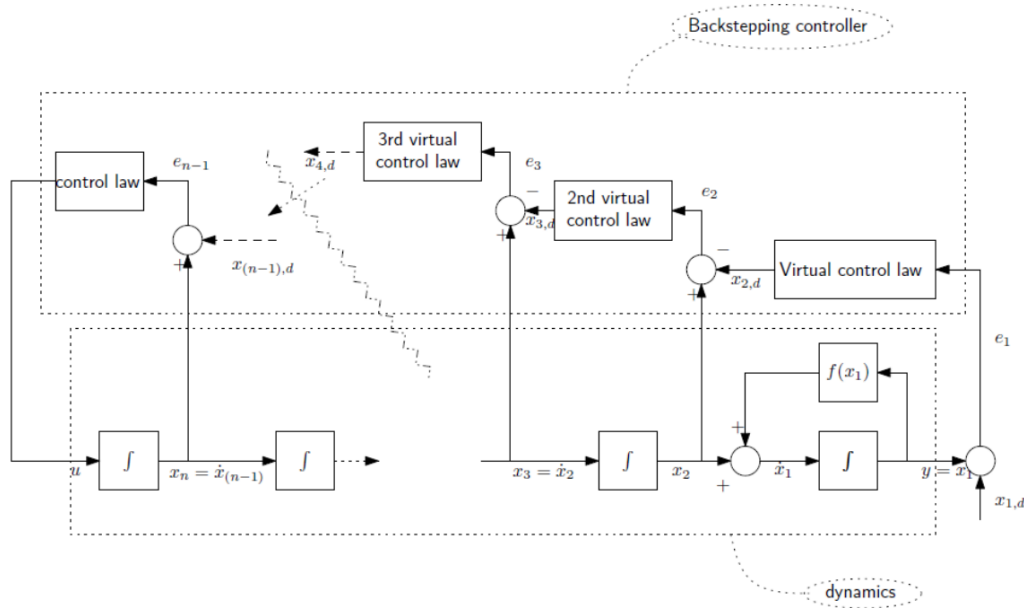
and the intermediate virtual controls are

$$\alpha_i = \frac{1}{g_{i-1}}(-g_{i-1}z_{i-1} - c_i z_i - f_i + \dot{\alpha}_{i-1})$$

At the final stage, the physical actuator command is

$$u = \alpha_n = \frac{1}{g_n}(-g_{n-1}z_{n-1} - c_n z_n - f_n + \dot{\alpha}_{n-1})$$

where the stabilizing gains c_n are positive. This recursive expression summarizes the complete strict-feedback backstepping construction.



9.12 Block-Diagram

The block diagram provides a physical interpretation of the recursion. The output error e_1 enters the first virtual-control law and generates the desired value $x_{2,d}$. The difference between the actual state x_2 and its desired value $x_{2,d}$ defines the next error e_2 . A subsequent virtual-control law generates the desired value for the next state; its mismatch defines e_3 , and the process continues until the final control law produces the physical actuator command u . Thus each outer state requests a target from the next inner state, and the chain continues until a realizable actuator input is obtained.